

COMPUTER VISION

Chapter 4: Feature Detection and Matching

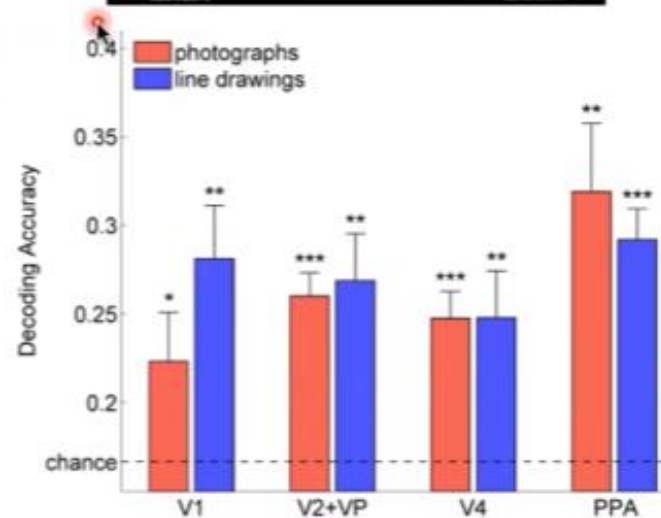
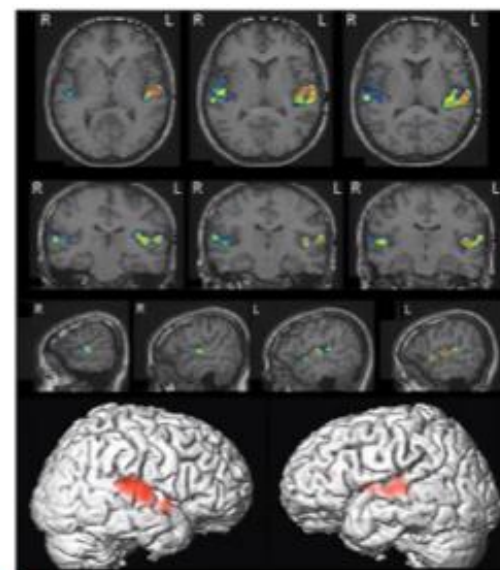
Lecturer: Dr. Do Huu Quan

What we will learn today

- **Edge detection**
- Image Gradients
- A simple edge detector
- Sobel edge detector
- Canny edge detector
- Hough Transform



- (A) Cave painting at Chauvet, France, about 30,000 B.C.;
- (B) Aerial photograph of the picture of a monkey as part of the Nazca Lines geoglyphs, Peru, about 700 – 200 B.C.;
- (C) Shen Zhou (1427-1509 A.D.): Poet on a mountain top, ink on paper, China;
- (D) Line drawing by 7-year old I. Lleras (2010 A.D.).



Walther, Chai, Caddigan, Beck & Fei-Fei, *PNAS*, 2011

Edge detection

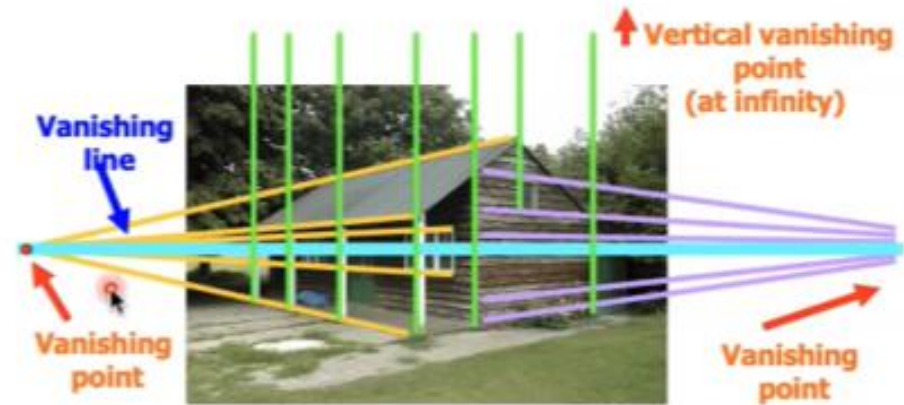
- **Goal:** Identify sudden changes (discontinuities) in an image
 - Intuitively, most semantic and shape information from the image can be encoded in the edges
 - More compact than pixels



Source: D. Lowe 

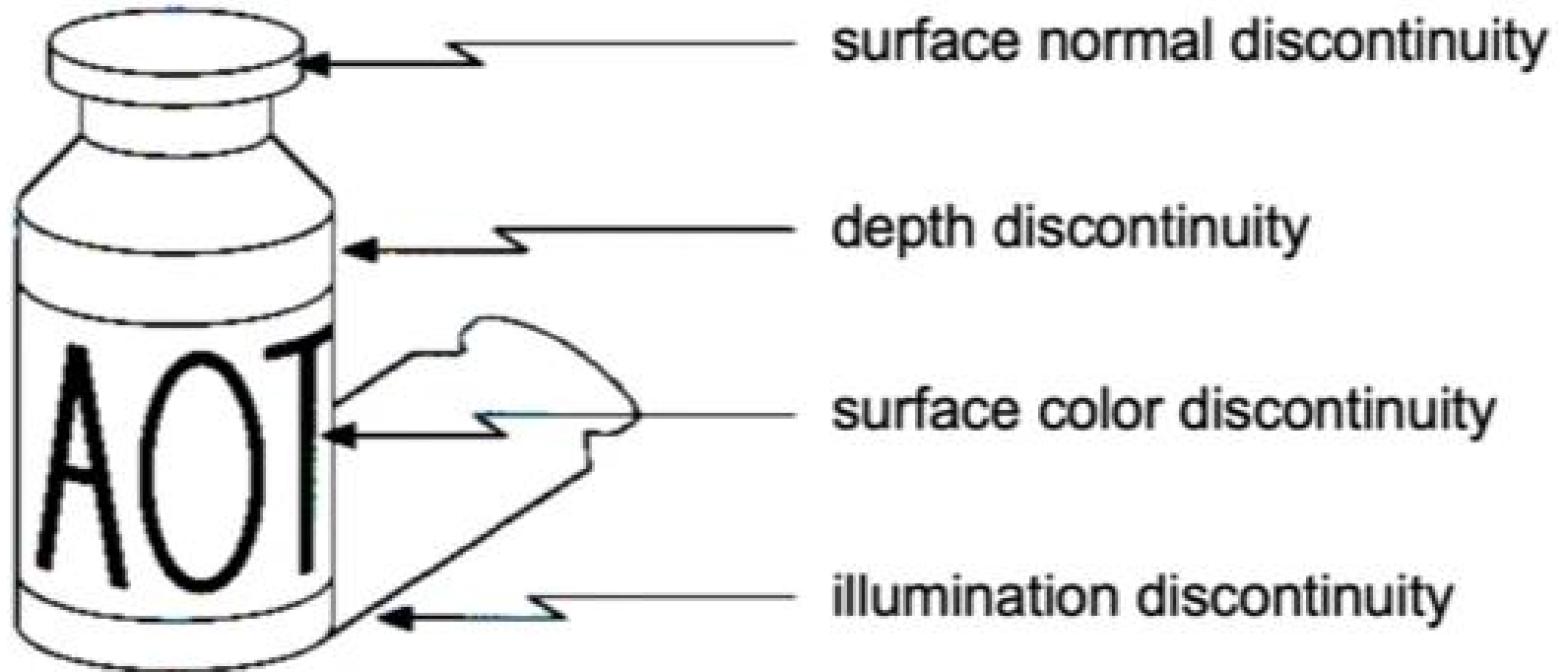
Why do we care about edges?

- Extract information, recognize objects
- Recover geometry and viewpoint



Source: J. Hayes

Origin of Edges



- Edges are caused by a variety of factor

What we will learn today

- Edge detection
- **Image Gradients**
- A simple edge detector
- Sobel edge detector
- Canny edge detector
- Hough Transform

Derivatives in 1D

$$\frac{df}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x) - f(x - \Delta x)}{\Delta x} = f'(x) = f_x$$

Types of discrete derivative in 1D

Backward $\frac{df}{dx} = f(x) - f(x-1) = f'(x)$

Forward $\frac{df}{dx} = f(x) - f(x+1) = f'(x)$

Central $\frac{df}{dx} = f(x+1) - f(x-1) = f'(x)$

Discrete derivative in 2D

Given function $f(x, y)$

Gradient vector $\nabla f(x, y) = \begin{bmatrix} \frac{\partial f(x, y)}{\partial x} \\ \frac{\partial f(x, y)}{\partial y} \end{bmatrix} = \begin{bmatrix} f_x \\ f_y \end{bmatrix}$

Gradient magnitude $|\nabla f(x, y)| = \sqrt{f_x^2 + f_y^2}$

Gradient direction $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

2D discrete derivative filters

What does this filter do?

$$\frac{1}{3} \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

2D discrete derivative filters

What about this filter?

$$\frac{1}{3} \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

2D discrete derivative - example

$$I = \begin{bmatrix} 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \end{bmatrix}$$

2D discrete derivative - example

What happens when we apply
this filter?

$$I = \begin{bmatrix} 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \end{bmatrix}$$

$$\frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

2D discrete derivative - example

What happens when we apply this filter?

$$I = \begin{bmatrix} 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \end{bmatrix}$$

$$\frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

$$I_y = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

2D discrete derivative - example

Now let's try the other filter!

$$I = \begin{bmatrix} 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \end{bmatrix}$$

$$\frac{1}{3} \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

2D discrete derivative - example

What happens when we apply this filter?

$$I = \begin{bmatrix} 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \\ 10 & 10 & 20 & 20 & 20 \end{bmatrix}$$

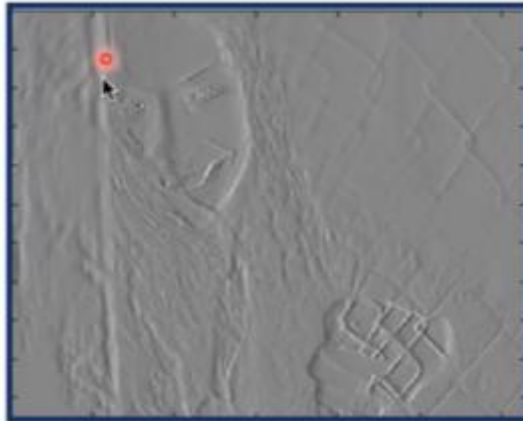
$$\frac{1}{3} \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

$$I_x = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 10 & 10 & 0 & 0 \\ 0 & 10 & 10 & 0 & 0 \\ 0 & 10 & 10 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

3x3 image gradient filters

$$\frac{1}{3} \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\frac{1}{3} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



What we will learn today

- Edge detection
- Image Gradients
- **A simple edge detector**
- Sobel edge detector
- Canny edge detector
- Hough Transform

Characterizing edges

- An edge is a place of rapid change in the image intensity function

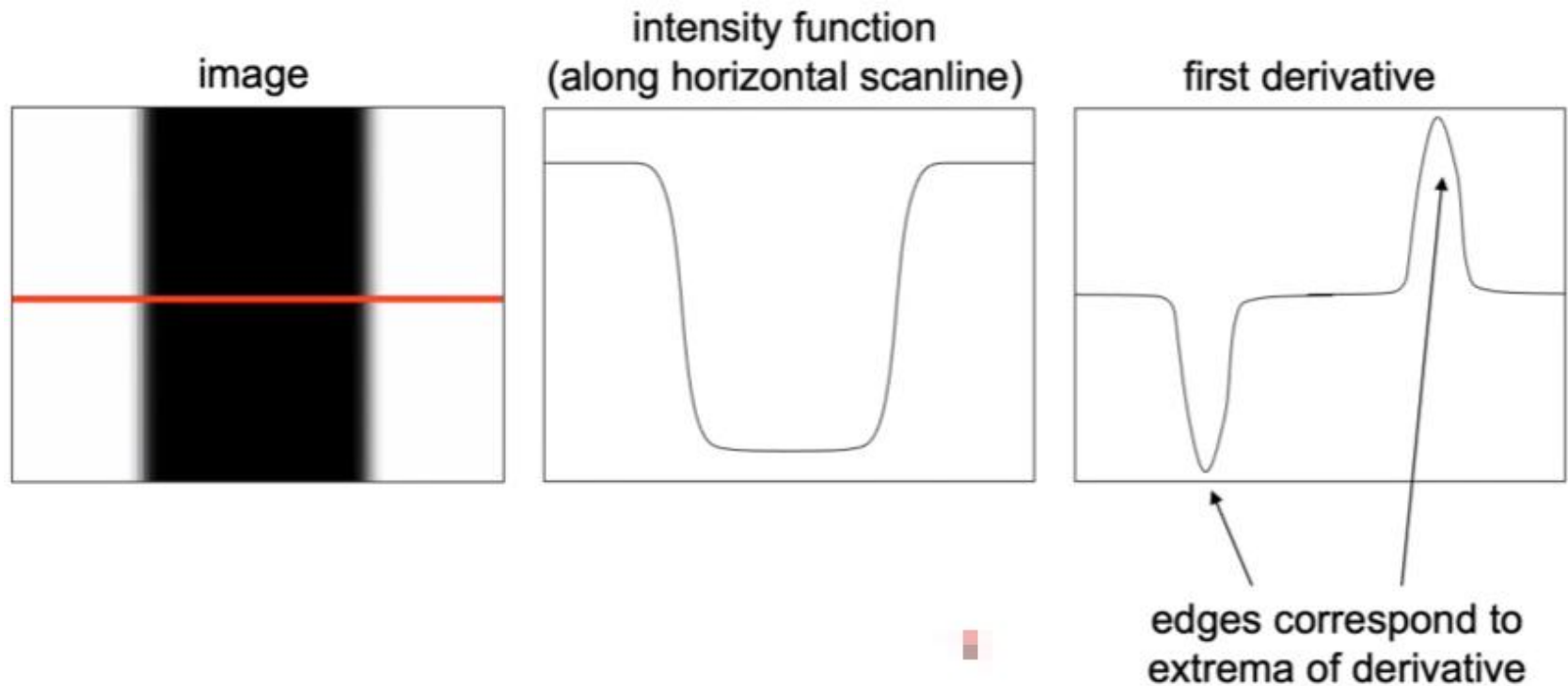
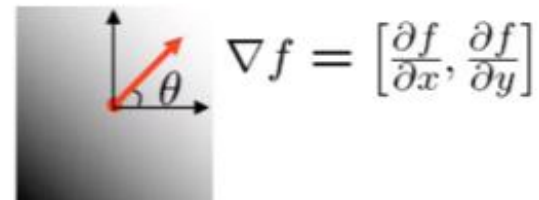
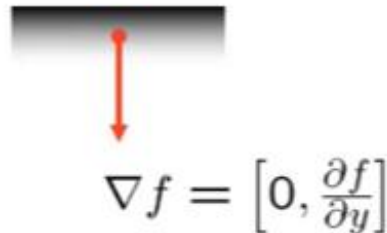
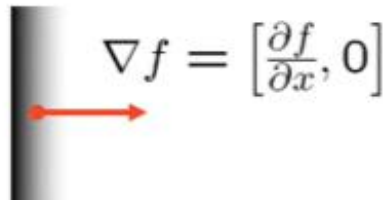


Image gradient

- The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$



The gradient vector points in the direction of most rapid increase in intensity

The gradient direction is given by $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

- how does this relate to the direction of the edge?

The *edge strength* is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Finite differences: Example

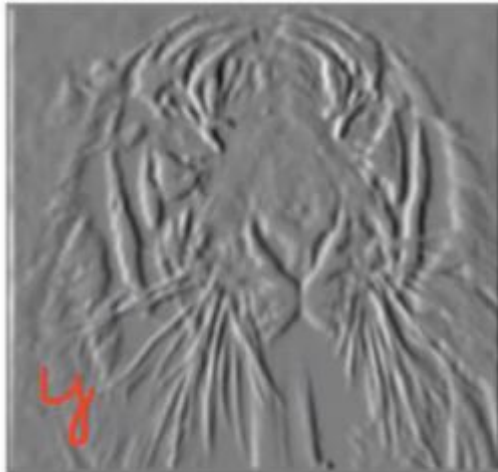
Original
Image



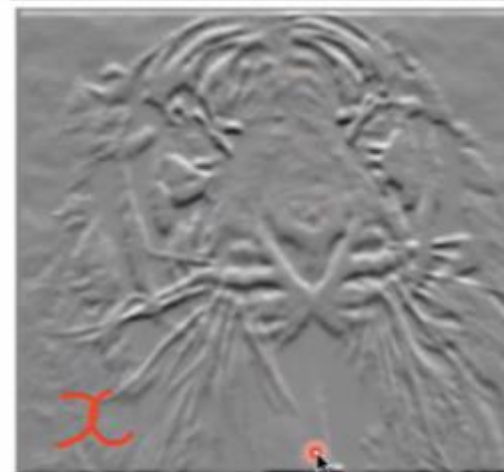
Gradient
magnitude



x-direction

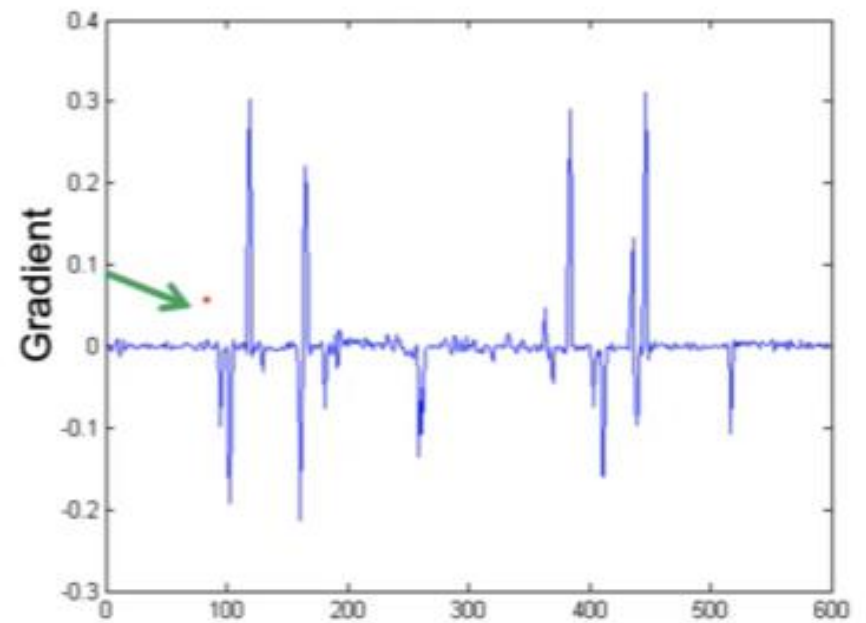
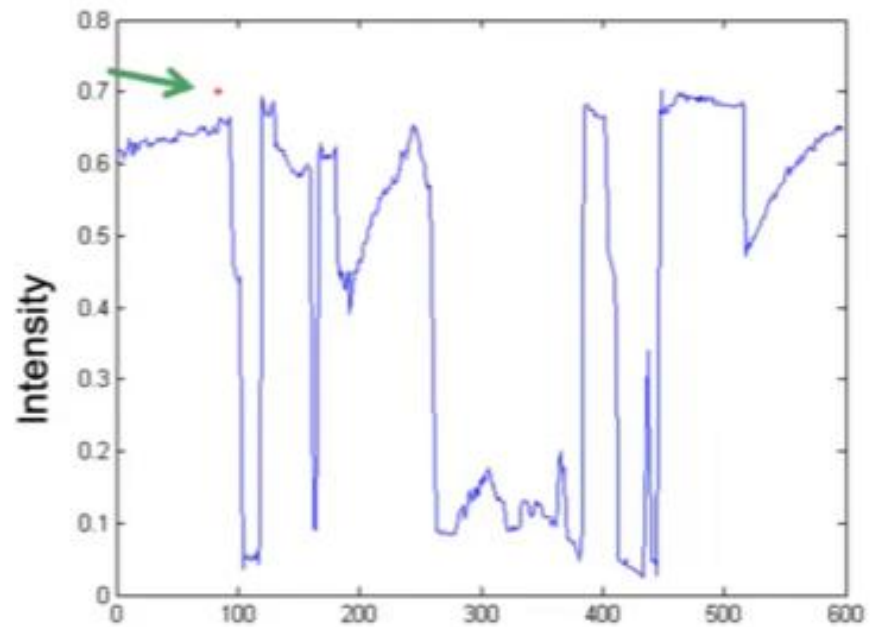
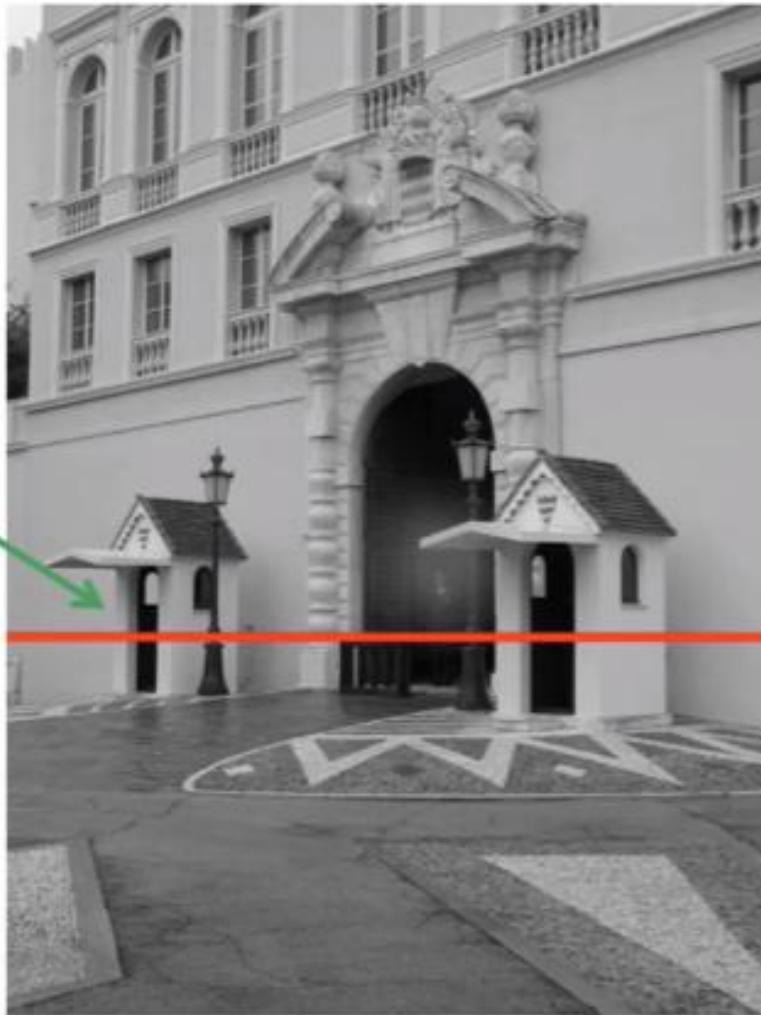


y-direction



- Which one is the gradient in the x-direction? How about y-direction?

Intensity profile

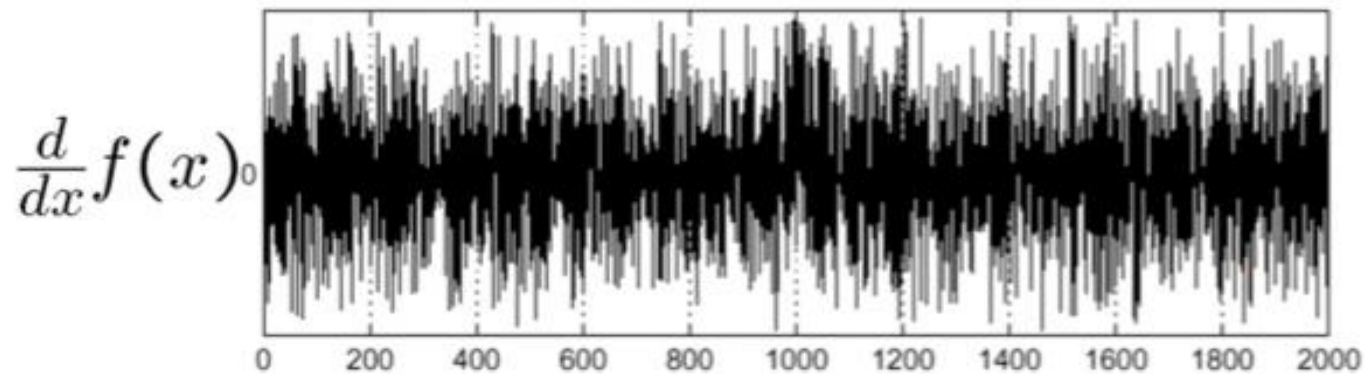
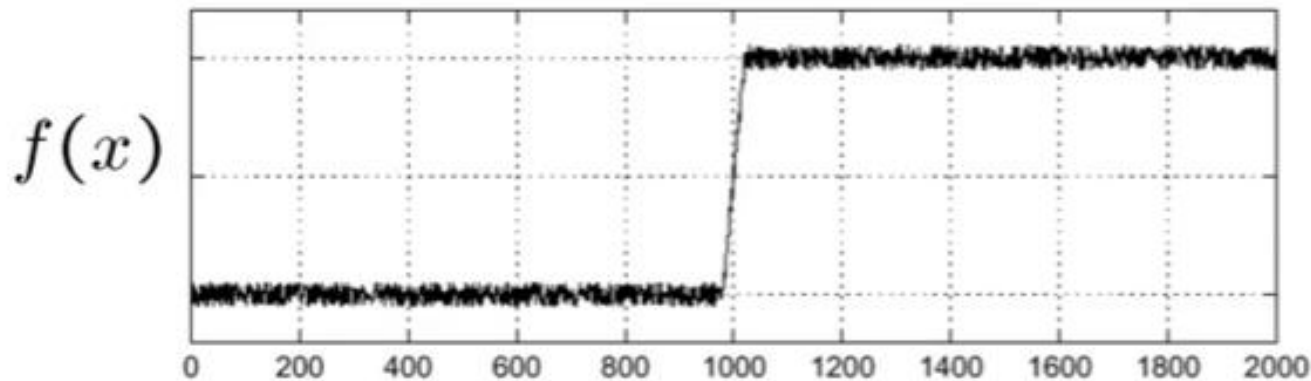


Source: D. Hoiem

Effects of noise

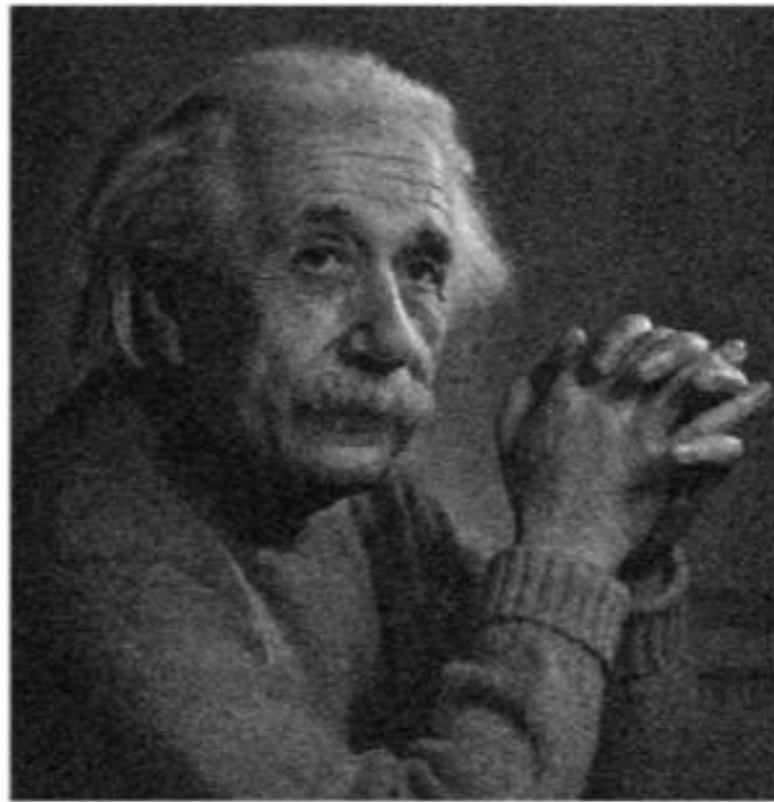
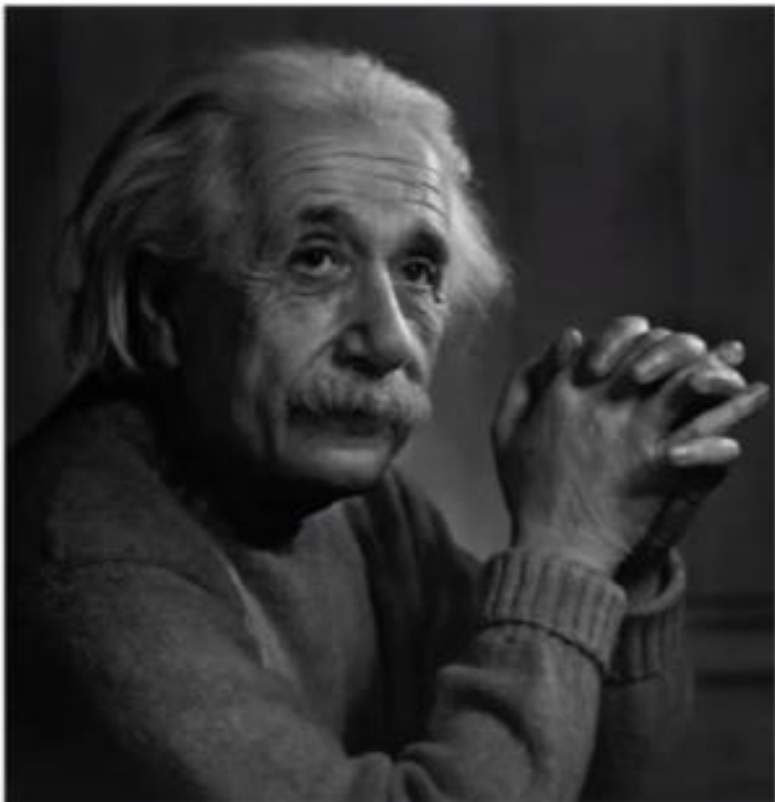
Consider a single row or column of the image

- Plotting intensity as a function of position gives a signal



Where is the edge?

Effects of noise



Effects of noise

- Finite difference filters respond strongly to noise
 - Image noise results in pixels that look very different from their neighbors
 - Generally, the larger the noise the stronger the response
- What is to be done?
 - Smoothing the image should help, by forcing pixels different to their neighbors (=noise pixels?) to look more like neighbors

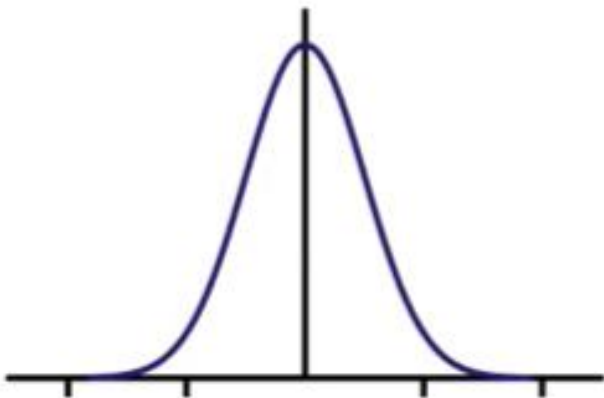
Smoothing with different filters

- Mean smoothing

$$\begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$[1 \quad 1 \quad 1]$$

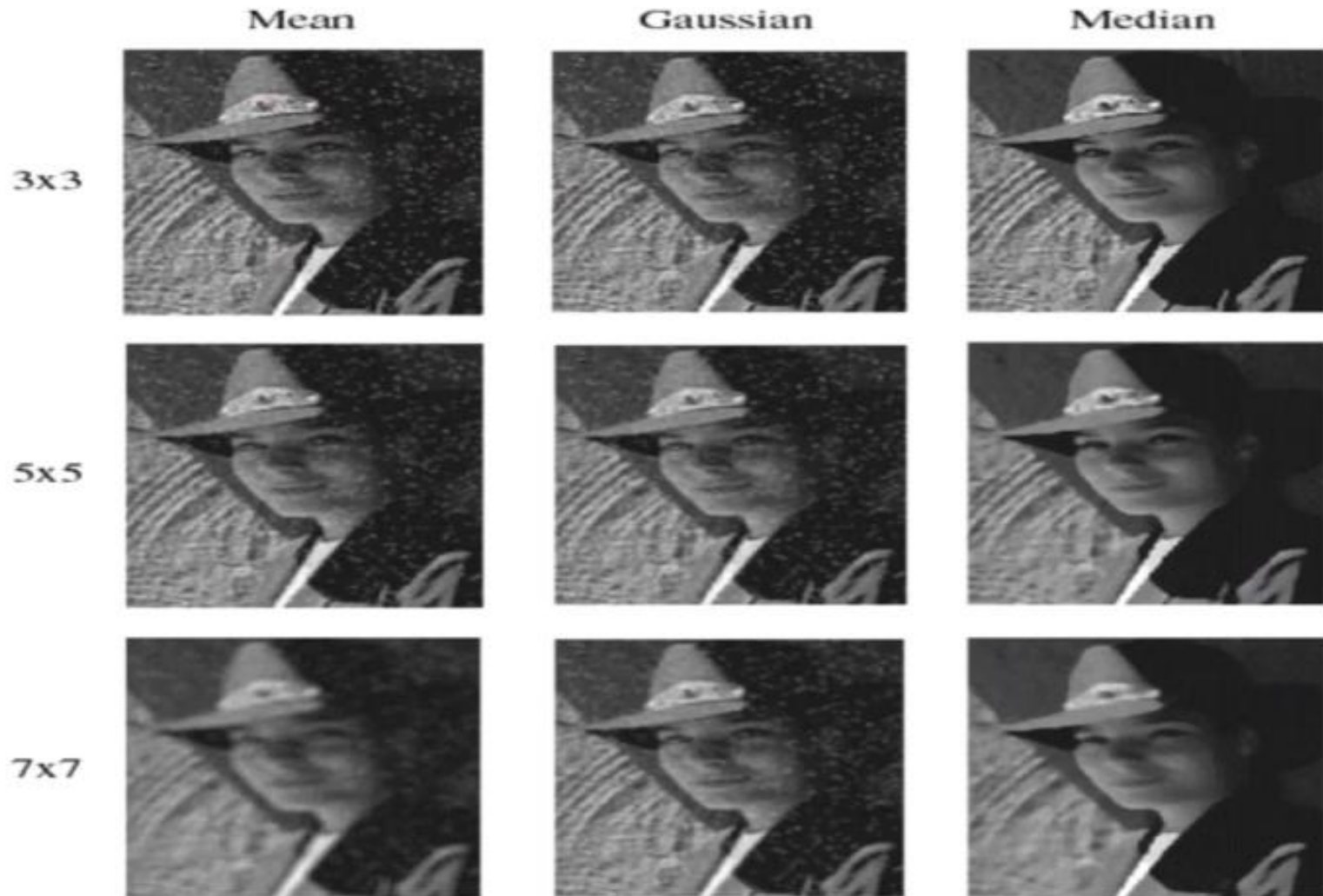
- Gaussian (smoothing * derivative)



$$\begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

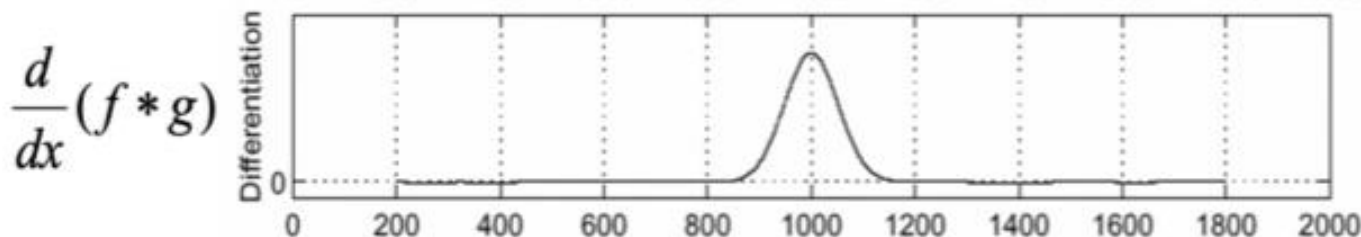
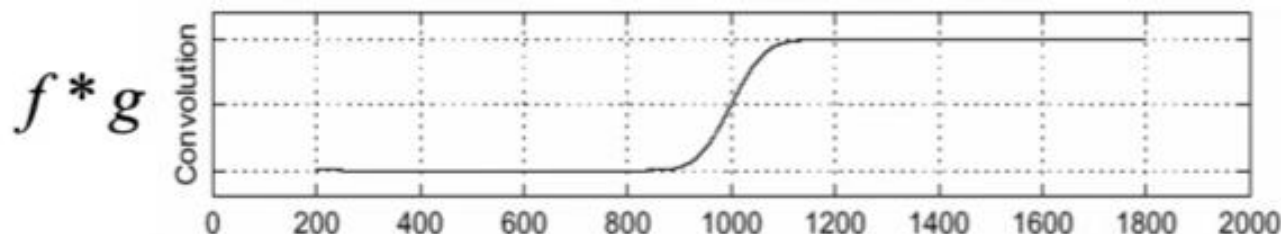
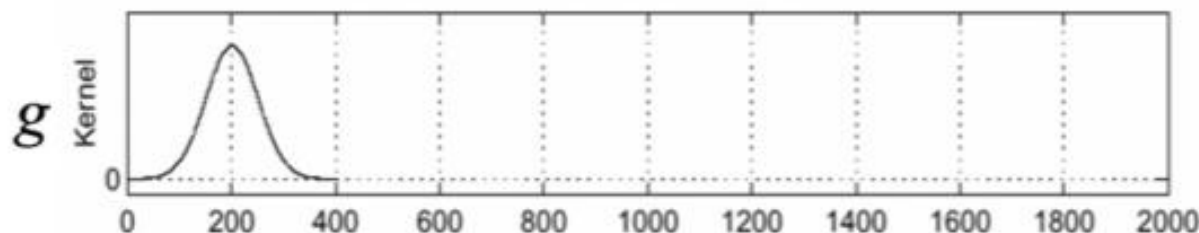
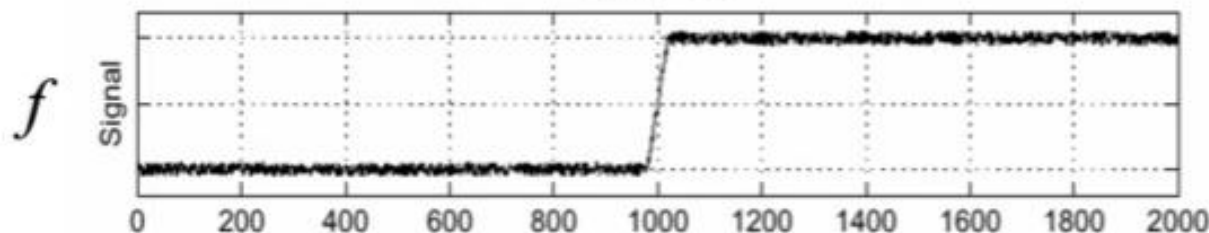
$$[1 \quad 2 \quad 1]$$

Smoothing with different filters



Solution: smooth first

Sigma = 50



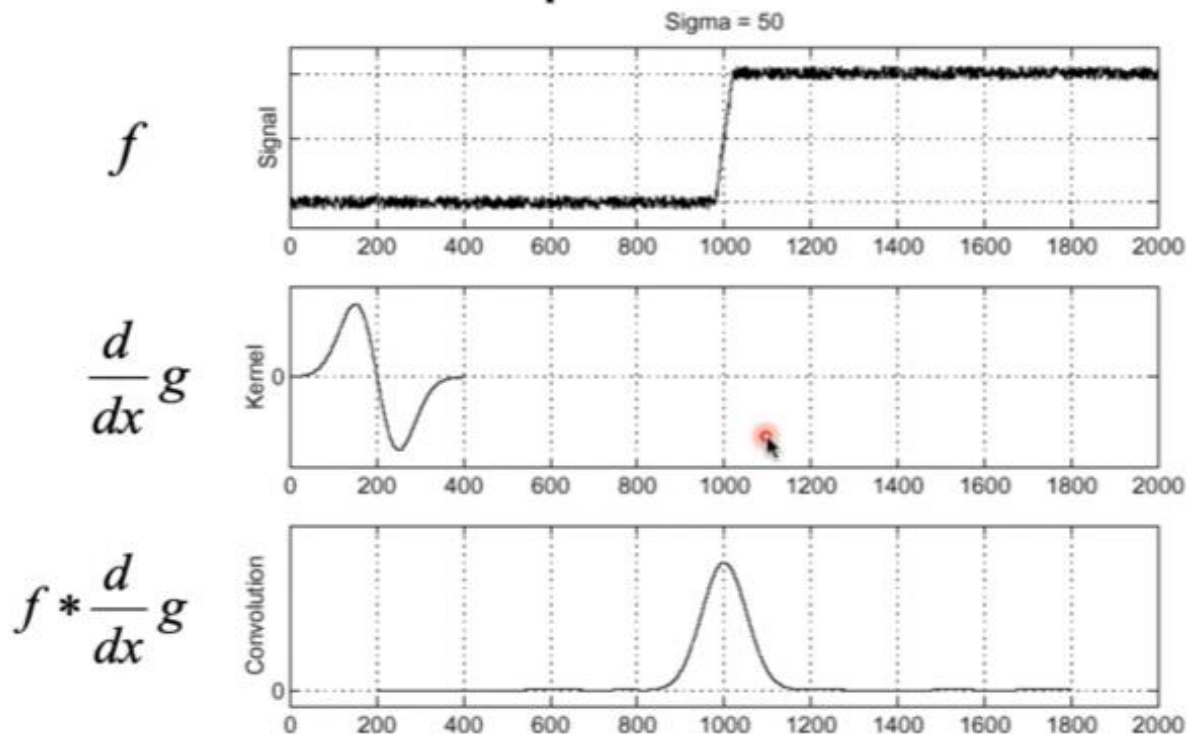
To find edges, look for peaks in $\frac{d}{dx}(f * g)$

Derivative theorem of convolution

- This theorem gives us a very useful property:

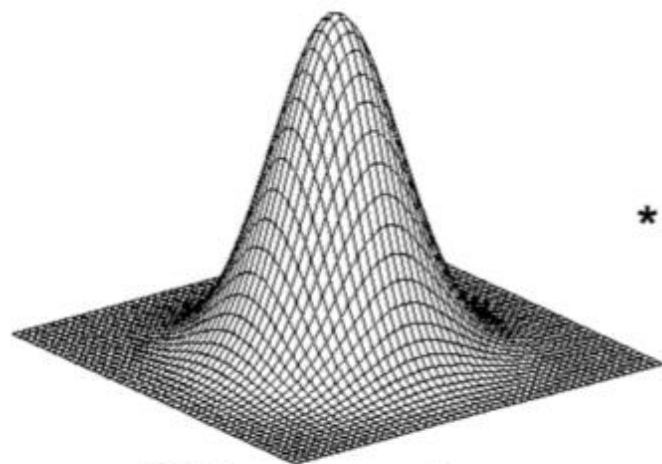
$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$$

- This saves us one operation:



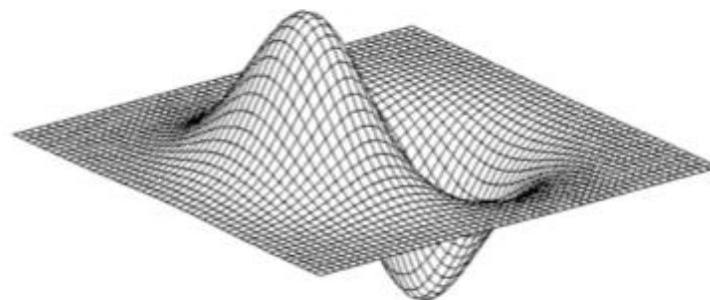
Source: S. Seitz

Derivative of Gaussian filter



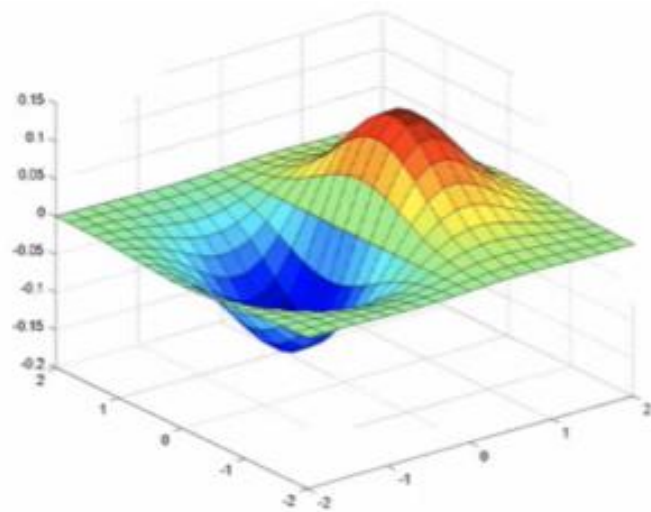
2D-gaussian

$$* \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} =$$

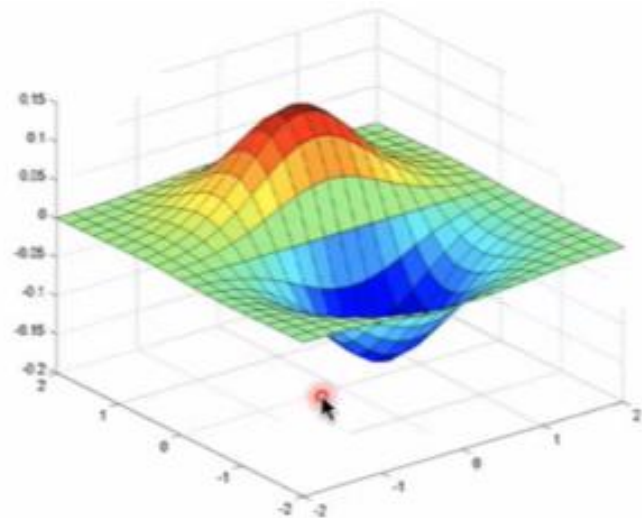
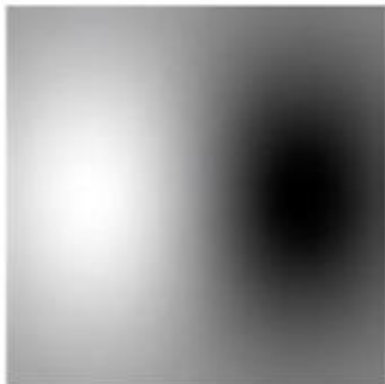


x - derivative

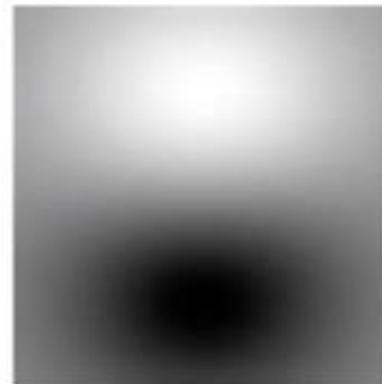
Derivative of Gaussian filter



x-direction



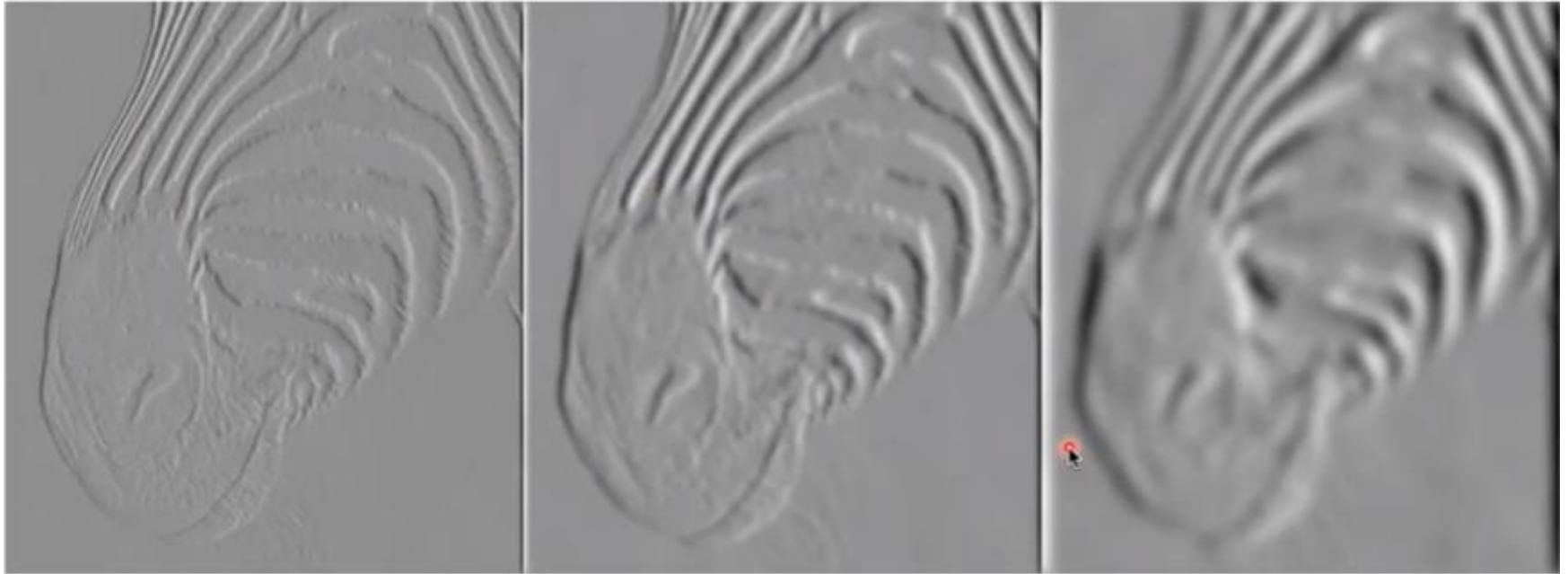
y-direction



Derivative of Gaussian filter



Tradeoff between smoothing at different scales



1 pixel

3 pixels

7 pixels

Smoothed derivative removes noise, but blurs edge.
Also finds edges at different “scales”.

Designing an edge detector

- Criteria for an “optimal” edge detector:
 - **Good detection:** the optimal detector must minimize the probability of false positives (detecting spurious edges caused by noise), as well as that of false negatives (missing real edges)
 - **Good localization:** the edges detected must be as close as possible to the true edges
 - **Single response:** the detector must return one point only for each true edge point; that is, minimize the number of local maxima around the true edge



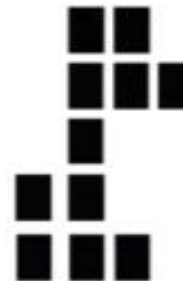
True edge



Poor robustness to noise



Poor localization



Too many responses

What we will learn today

- Edge detection
- Image Gradients
- A simple edge detector
- **Sobel edge detector**
- Canny edge detector
- Hough Transform

Sobel Operator

- Uses two 3x3 kernels which are convolved with the original image to calculate approximations of the derivatives
- one for horizontal changes, and one for vertical

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Sobel Operator

- Smoothing + differentiation

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} +1 & 0 & -1 \end{bmatrix}$$

Gaussian smoothing

differentiation

Sobel Operator

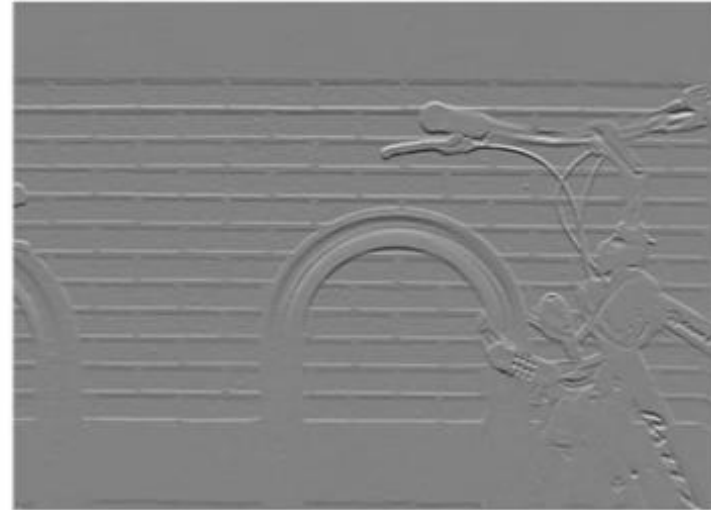
- Magnitude:

$$\mathbf{G} = \sqrt{\mathbf{G}_x^2 + \mathbf{G}_y^2}$$

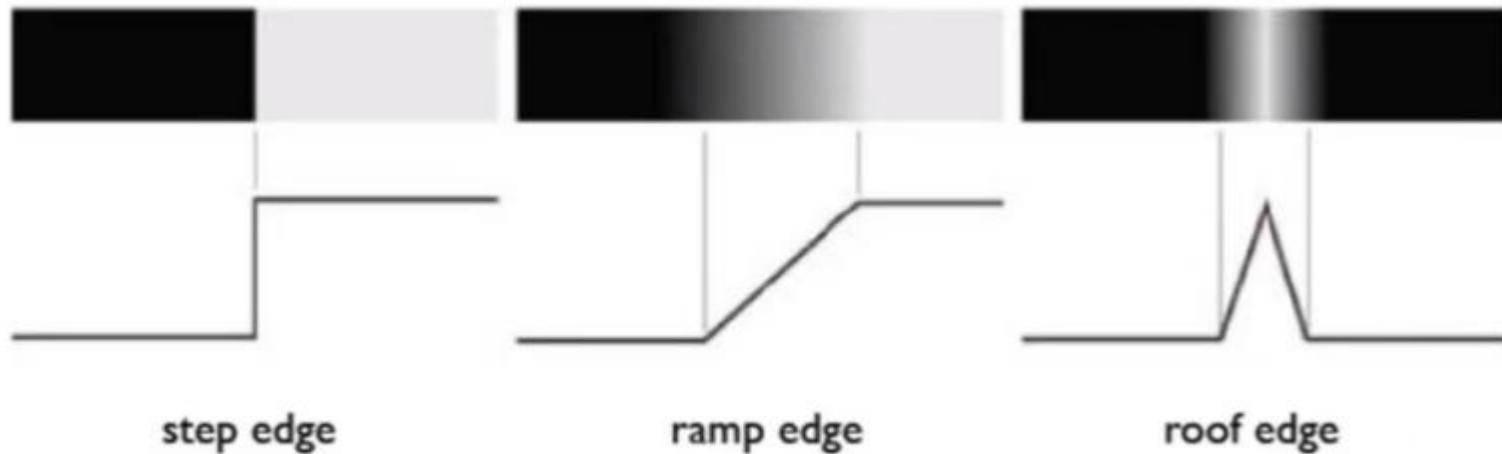
- Angle or direction of the gradient:

$$\Theta = \text{atan}\left(\frac{\mathbf{G}_y}{\mathbf{G}_x}\right)$$

Sobel Filter example



Sobel Filter Problem



- Poor localization (Trigger response in multiple adjacent pixels)
- Thresholding value favors certain directions over others
 - Can miss oblique edges more than horizontal or vertical edges
 - False negatives

What we will learn today

- Edge detection
- Image Gradients
- A simple edge detector
- Sobel edge detector
- **Canny edge detector**
- Hough Transform

Canny edge detector

- Probably the most widely used edge detector in computer vision

J. Canny, [A computational Approach to Edge Detection](#), IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-714, 1986. -> 43.000 citations!

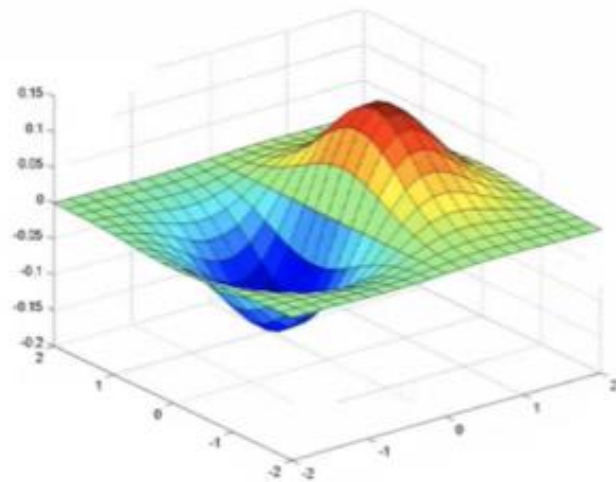
Demonstrator Image



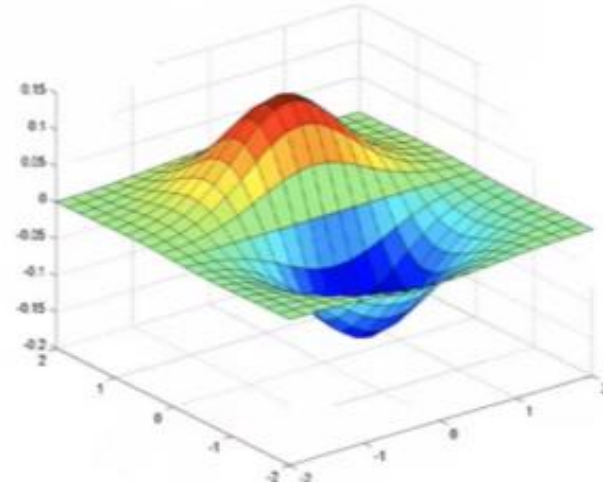
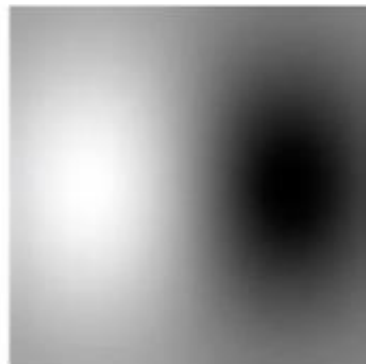
Canny edge detector

1. Filter image with x, y derivatives of Gaussian

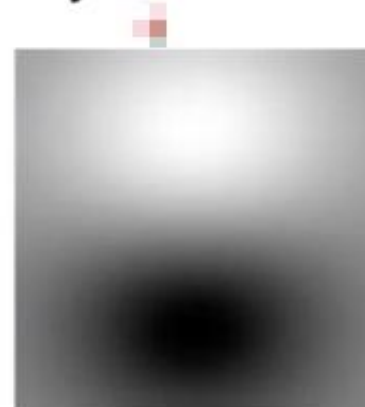
Derivative of Gaussian filter



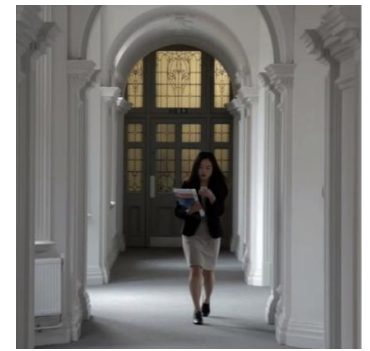
x-direction



y-direction



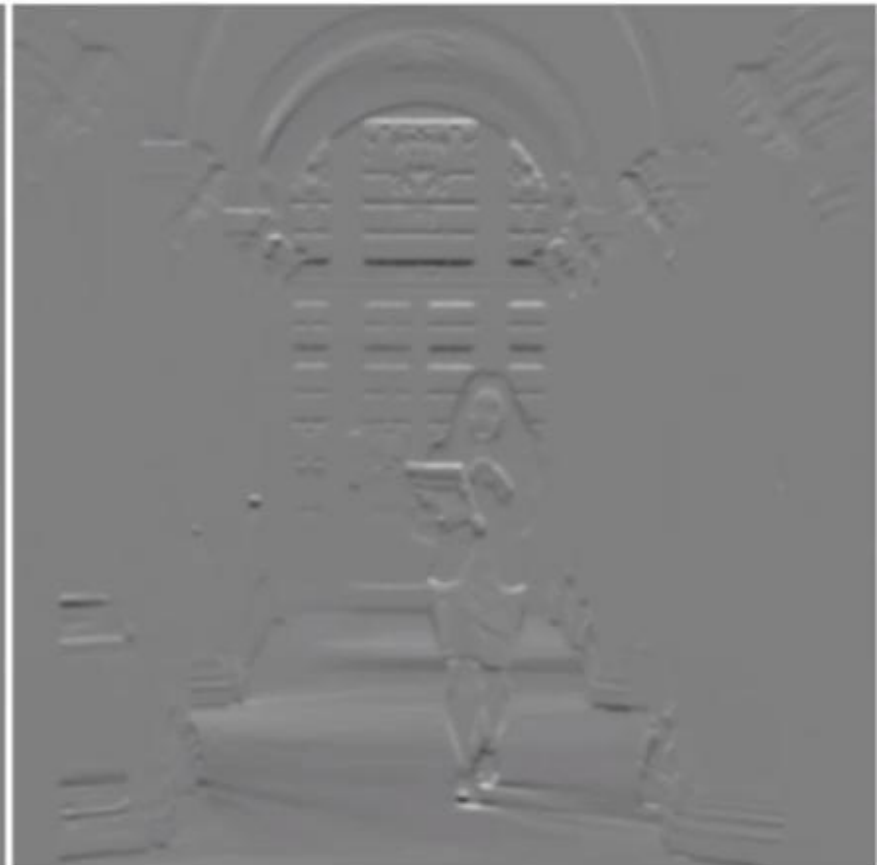
Compute Gradients



X Derivative of Gaussian



Y Derivative of Gaussian

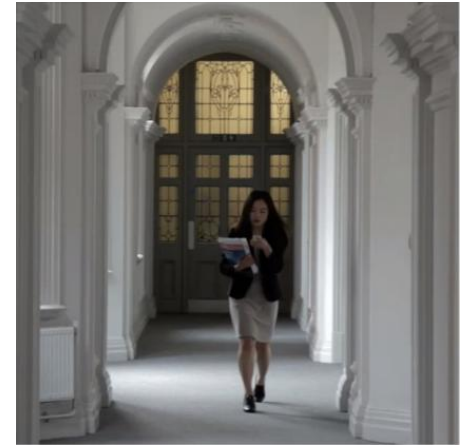


(x2 + 0.5 for visualization)

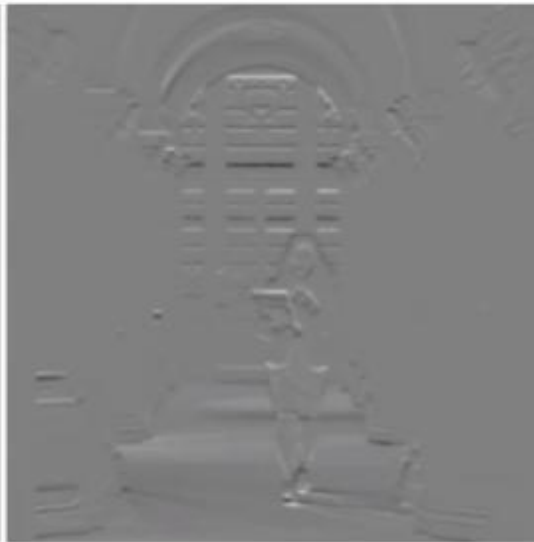
Canny edge detector

1. Filter image with x, y derivatives of Gaussian
2. Find magnitude and orientation of gradient

Compute Gradient Magnitude



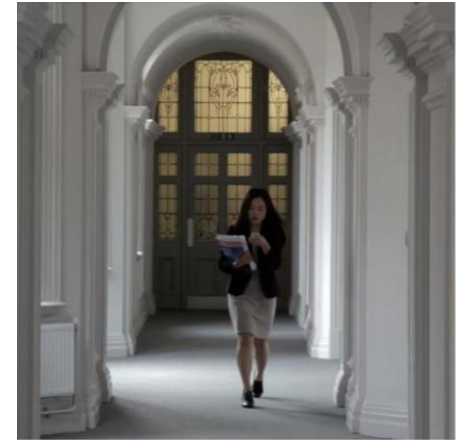
$\text{sqrt}(\text{XDerivOfGaussian}.^2 + \text{YDerivOfGaussian}.^2)$ = gradient magnitude



(x4 for visualization)

Compute Gradient Orientation

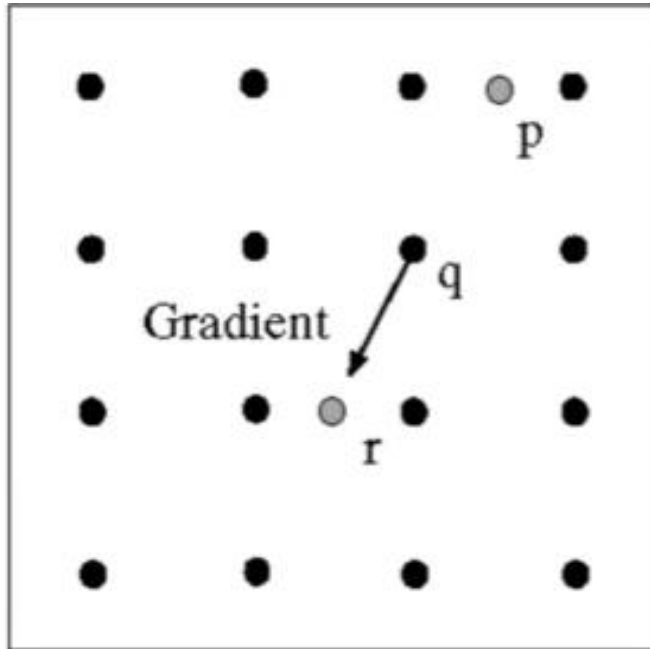
- Threshold magnitude at minimum level
- Get orientation via $\theta = \text{atan2}(g_y, g_x)$



Canny edge detector

1. Filter image with x, y derivatives of Gaussian
2. Find magnitude and orientation of gradient
3. Non-maximum suppression:
 - Thin multi-pixel wide “ridges” to single pixel width

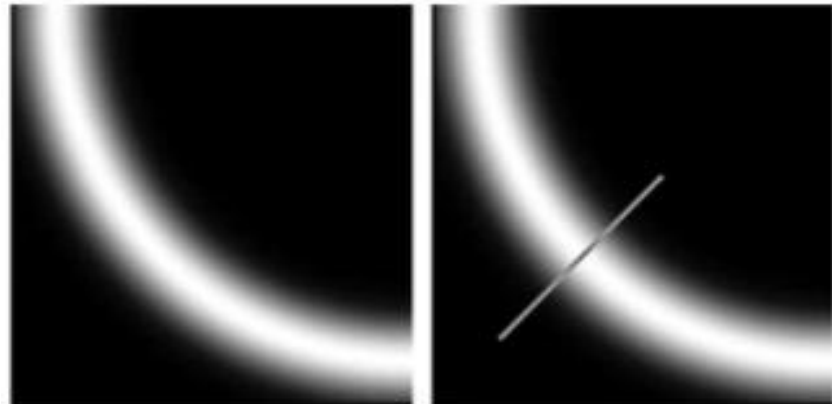
Non-maximum suppression for each orientation



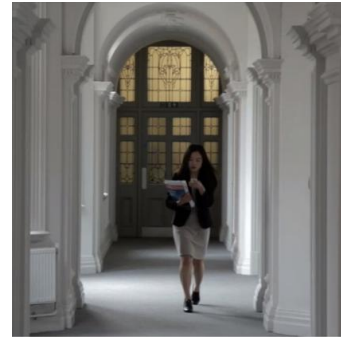
At pixel q :

We have a maximum if the value is larger than those at both p and at r

Interpolate along gradient direction to get these values

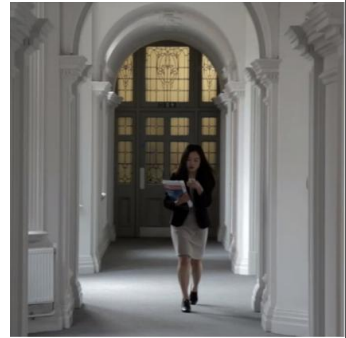


Before Non-max Suppression



Gradient magnitude (x4 for visualization)

After Non-max Suppression

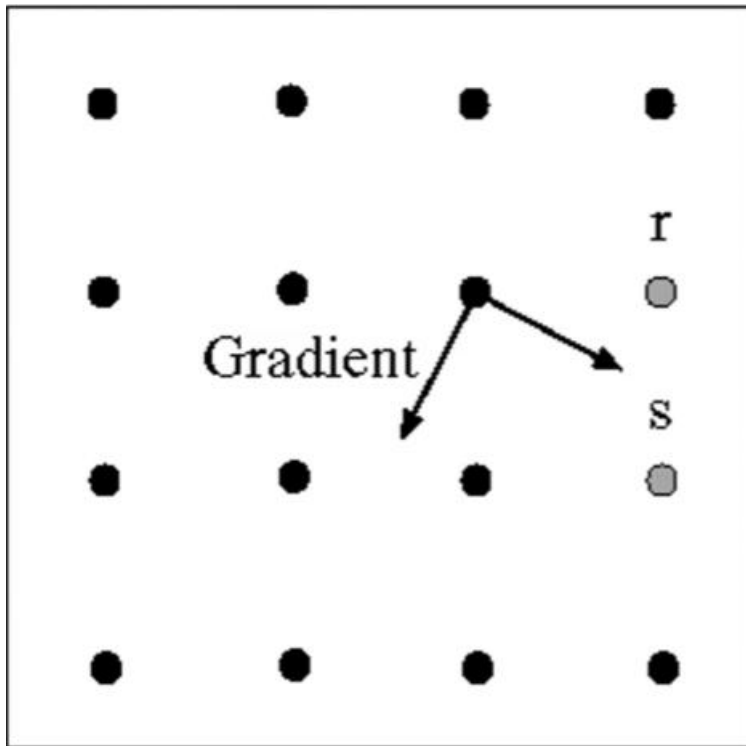


Gradient magnitude (x4 for visualization)

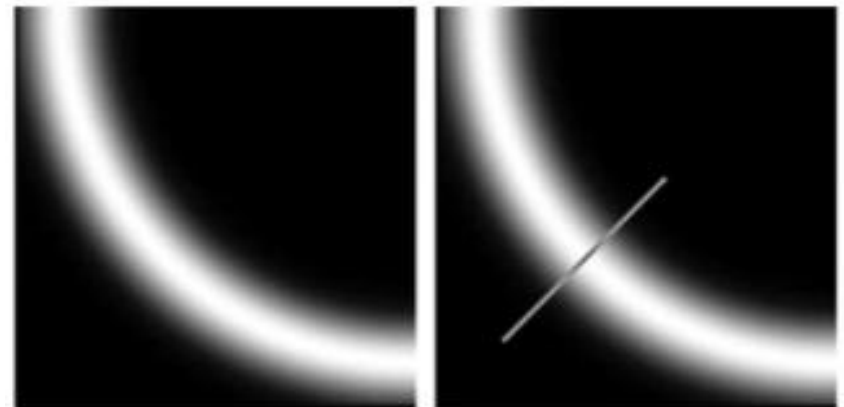
Canny edge detector

1. Filter image with x, y derivatives of Gaussian
2. Find magnitude and orientation of gradient
3. Non-maximum suppression:
 - Thin multi-pixel wide “ridges” to single pixel width
4. “Hysteresis” Thresholding

Edge linking



Assume the marked point is an edge point. Then we construct the tangent to the edge curve (which is normal to the gradient at that point) and use this to predict the next points (here either r or s).



“Hysteresis” Thresholding

- Two thresholds - high and low
- Grad. mag. $>$ high threshold? = strong edge
- Grad. mag. $<$ low threshold? noise
- In between = weak edge
- “Follow” edges starting from strong edge pixels
- Continue them into weak edges
- Connected components (Szeliski 3.3.4)



Final Canny Edges

$$\sigma = \sqrt{2}, \quad t_{low} = 0.05, \quad t_{high} = 0.1$$



Effect of σ (Gaussian kernel spread/size)



Original



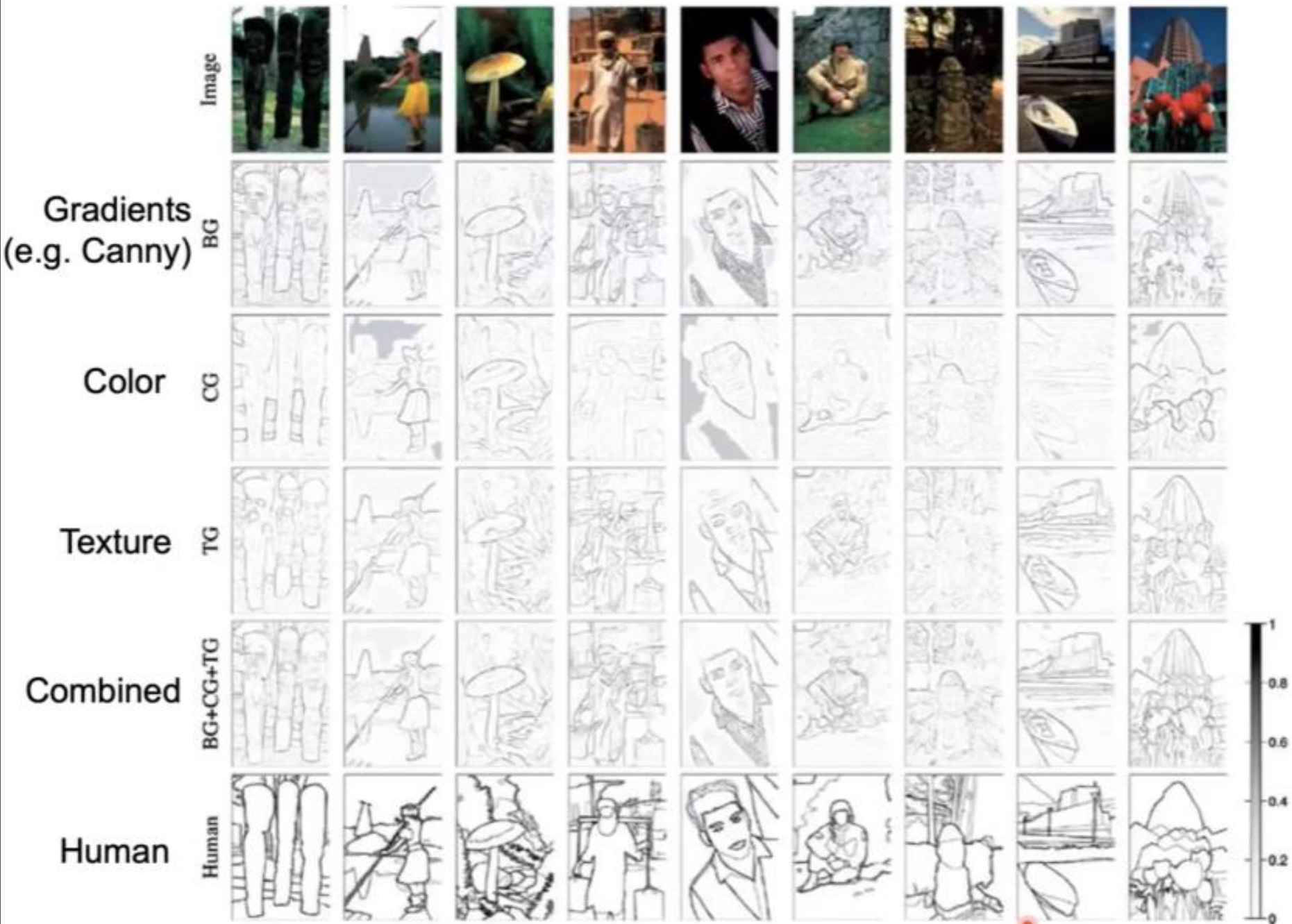
$\sigma = \sqrt{2}$



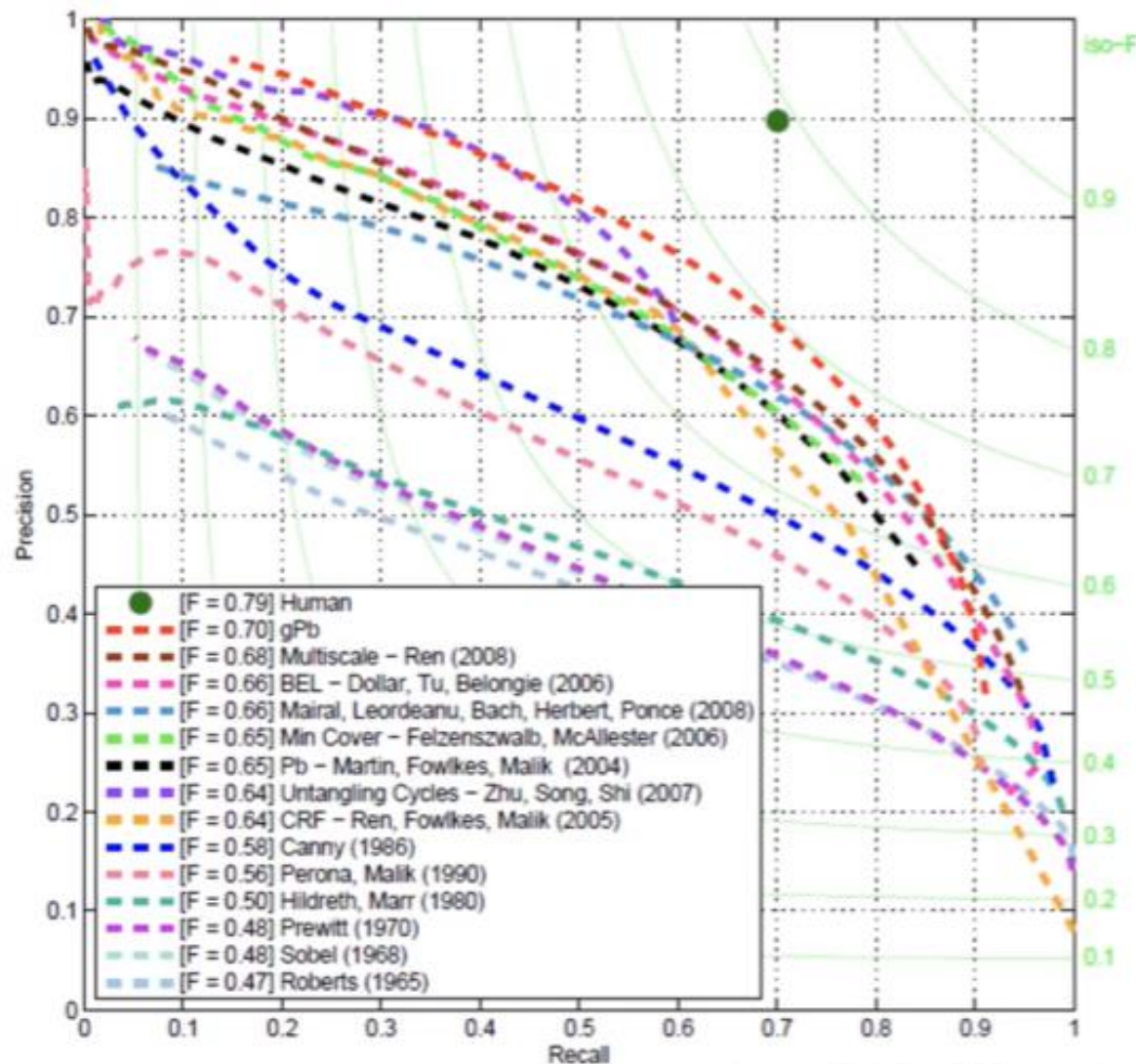
$\sigma = 4\sqrt{2}$

The choice of σ depends on desired behavior

- Large σ detects large scale edges
- Small σ detects fine features



45 years of boundary detection



Source: Arbelaez, Maire, Fowlkes, and Malik. TPAMI 2011 (pdf)

What we will learn today

- Edge detection
- Image Gradients
- A simple edge detector
- Sobel edge detector
- Canny edge detector
- **Hough Transform**

Intro to Hough transform

- The Hough transform (HT) can be used to detect lines.
- It was introduced in 1962 (Hough 1962) and first used to find lines in images a decade later (Duda 1972)
- The goal is to find the location of lines in images
- Caveat: Hough transform can detect lines, circles and other structures ONLY if their parametric equation is known.
- It can give robust detection under noise and partial occlusion.

Prior to Hough transform

- Assume that we have performed some edge detection, and a thresholding of the edge magnitude image.
- Thus, we have some pixels that many partially describe the boundary of some objects



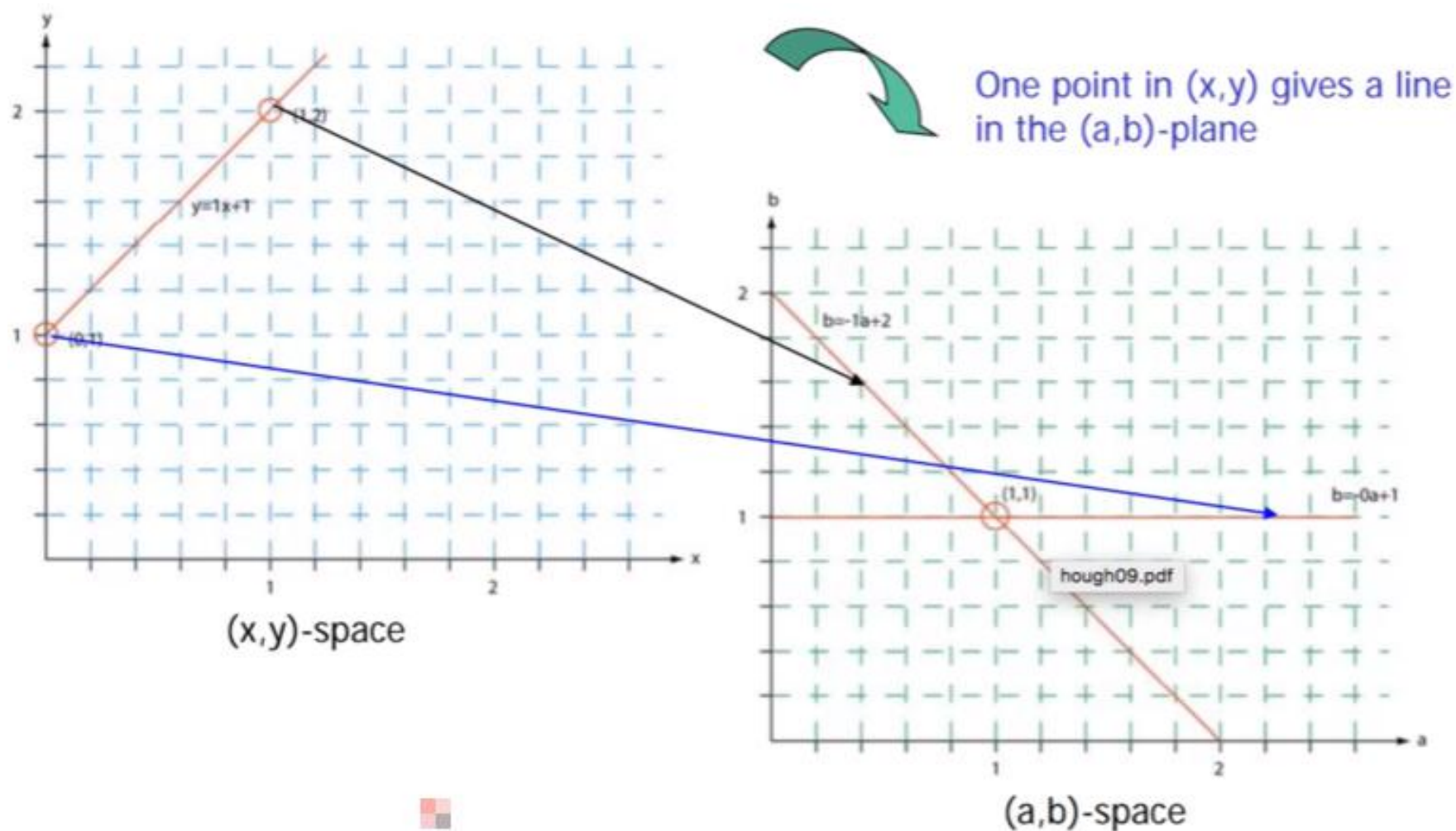
Detecting lines using Hough transform

- We wish to find sets of pixels that make up straight lines
- Consider a point of known coordinates $(x_i; y_i)$
 - There are many lines passing through the point $(x_i; y_i)$
- Straight lines that pass through that point have the form
$$y_i = a * x_i + b$$
 - Common to them is that they satisfy the equation for some set of parameters (a, b)

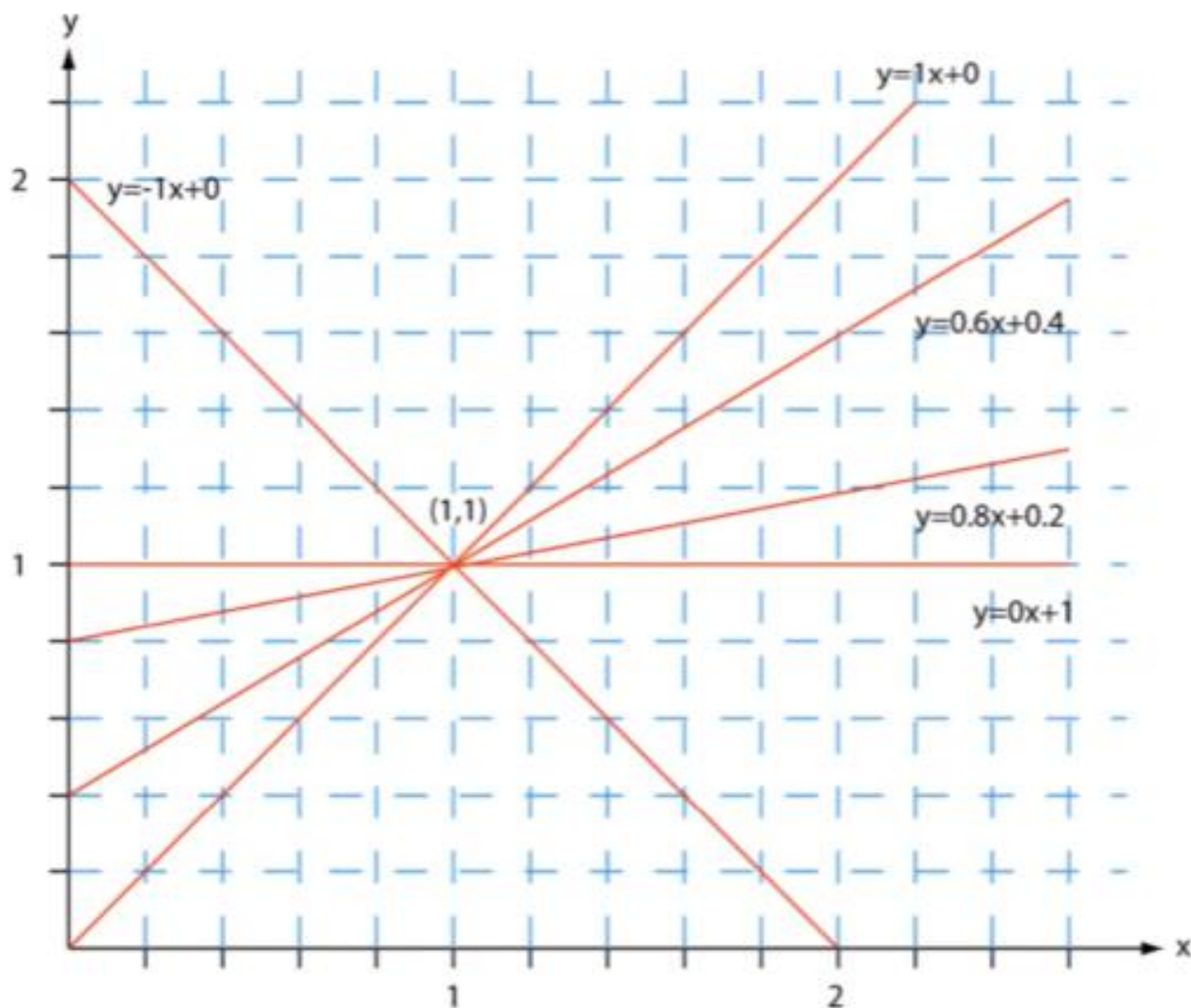
Detecting lines using Hough transform

- This equation can obviously be rewritten as follows:
 - $b = -a \cdot x_i + y_i$
 - We can now consider x and y as parameters
 - a and b as variables.
- This is a line in (a, b) space parameterized by x and y .
 - So a single point in x_1, y_1 - space gives a line in (a, b) space.
 - Another point (x_2, y_2) will give rise to another line (a, b) space.

Detecting lines using Hough transform



Detecting lines using Hough transform



Detecting lines using Hough transform

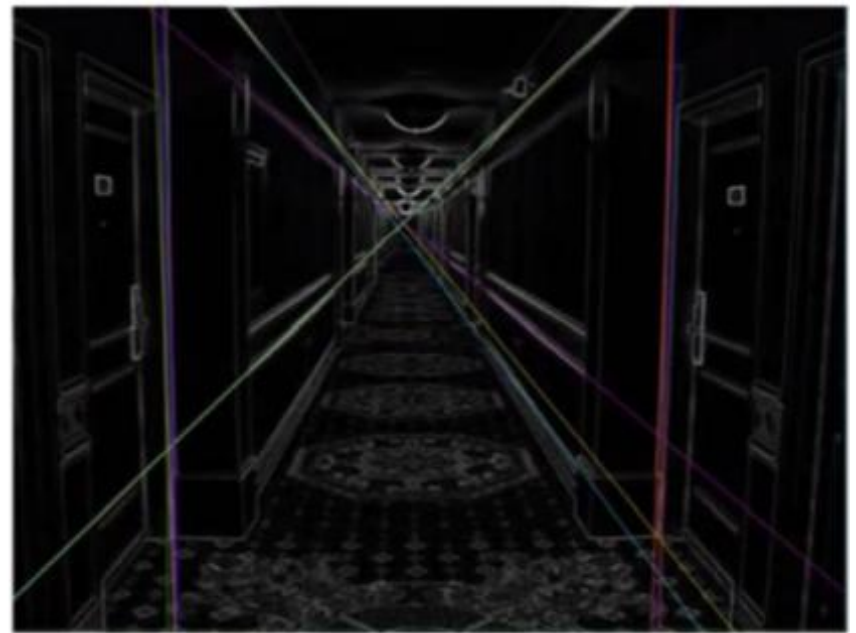
- Two points (x_1, y_1) and (x_2, y_2) define a line in the (x, y) plane.
- These two points give rise to two different lines in (a, b) space
- In (a, b) space these lines will intersect in a point (a', b')
- All points on the line defined by (x_1, y_1) and (x_2, y_2) in (x, y) space will parameterize lines that intersect in (a', b') in (a, b) space.

Algorithm for Hough transform

- Quantize the parameter space (a, b) by dividing it into cells.
- This quantized space is often referred to as the accumulator cells
- Count the number of times a line intersects a given cell.
 - For each pair of points (x_1, y_1) and (x_2, y_2) detected as an edge, find the intersection (a', b') in (a, b) space.
 - Increase the value of a cell in the range $[[a_{\min}, a_{\max}], [b_{\min}, b_{\max}]]$ that (a', b') belongs to.
 - Cells receiving more than a certain number of counts (also called “votes”) are assumed to correspond to lines in (x, y) space.

Out of Hough transform

- Here are the top 20 most voted lines in the image:

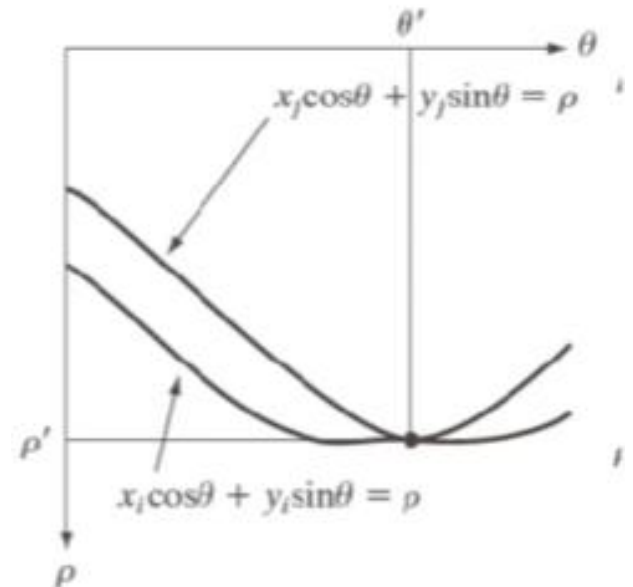
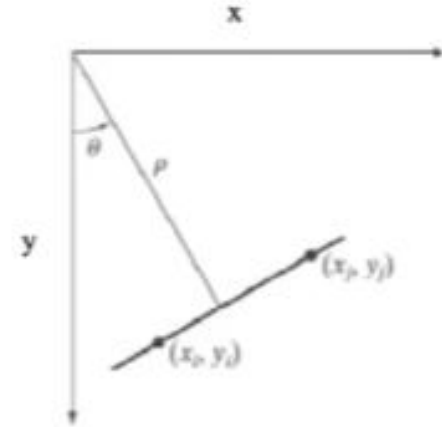


Other Hough transformations

- We can represent lines as polar coordinates instead of $y = a*x + b$
- Polar coordinate representation:
 - $x*\cos\theta + y*\sin\theta = \rho$
- Can you figure out the relationship between
 - (x, y) and (ρ, θ)

Other Hough transformations

- Note that lines in (x, y) space are not lines in (ρ, θ) space, unlike (a, b) space.
- A vertical line will have $\theta = 0$ and ρ equal to the intercept with the x - axis.
- A horizontal line will have $\theta = 90^\circ$ and ρ equal to the intercept with the y -axis.



Other Hough transformations

- Using the polar parameterization:

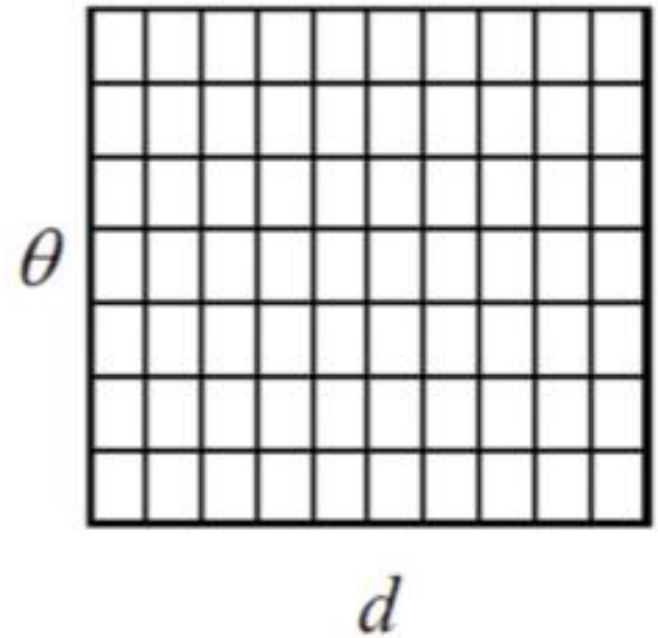
- $x \cdot \cos \theta + y \cdot \sin \theta = \rho$.

- Basic Hough transform algorithm

1. Initialize $H[d, \theta] = 0$
2. For each edge point (x, y) in the image for $\theta \in [0, \pi)$. //some quantization
 $H[d, \theta] += 1$.
3. Find the value(s) of (d, θ) where $H[d, \theta]$ is maximum.
4. The detected line in the image is given by

$$d = x \cdot \cos \theta + y \cdot \sin \theta$$

H : accumulator array (votes)

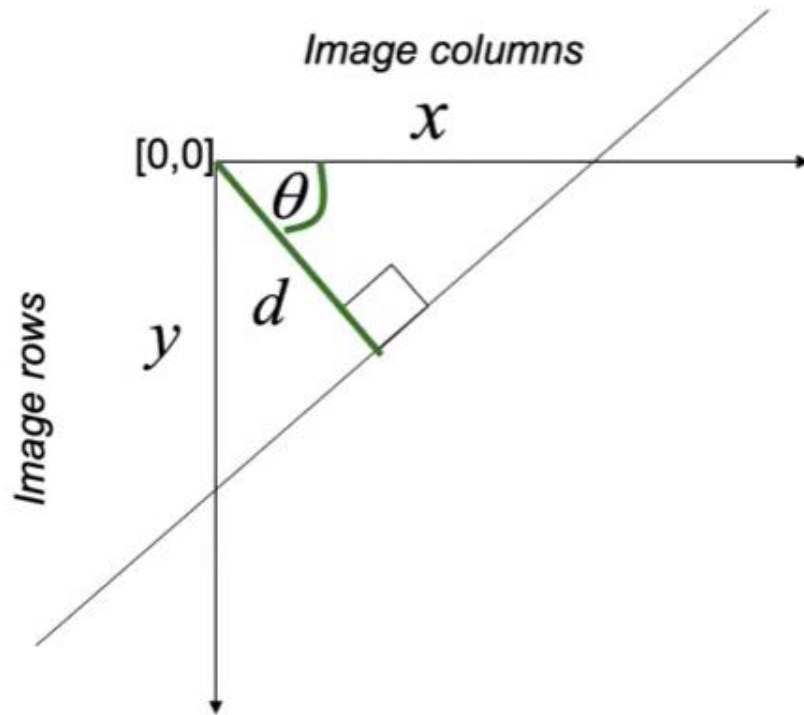


Video example

- <https://www.youtube.com/watch?v=4zHbl-fFIII>

Polar representation for lines

- Issues with usual (m, b) parameter space: can take on infinite values, undefined for vertical lines.



- d : perpendicular distance from line to origin
- θ : angle the perpendicular makes with the x-axis

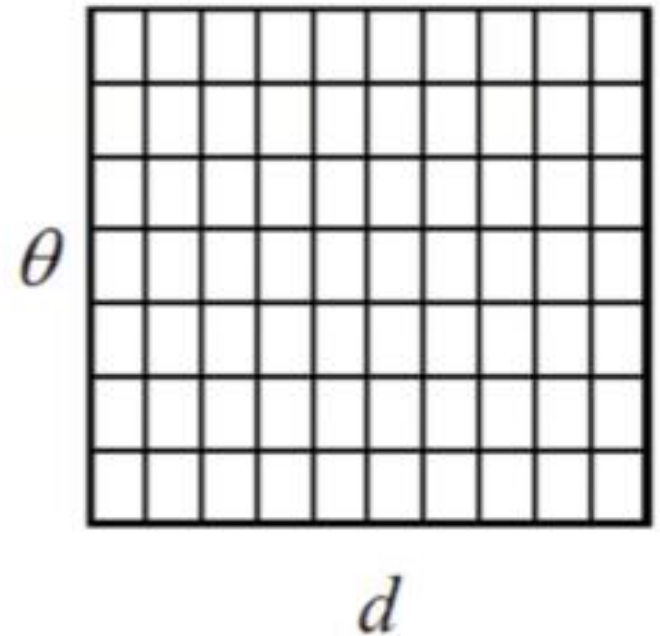
$$x \cos \theta - y \sin \theta = d$$

Point in image space --> Sinusoid segment in Hough space

Hough transform algorithm

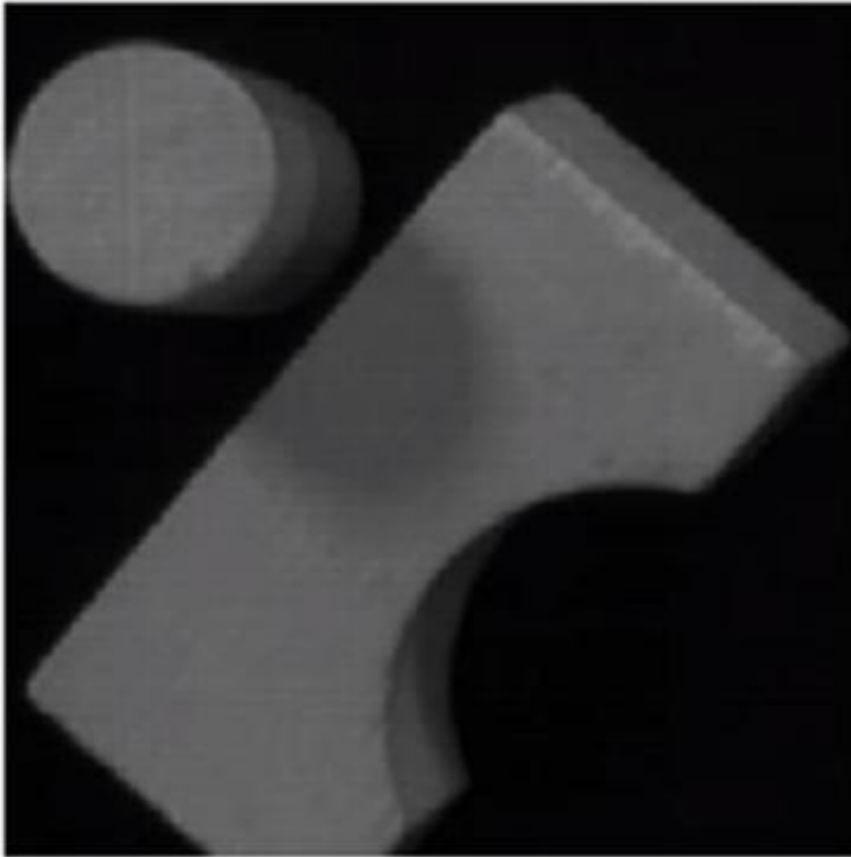
- Using the polar parameterization:
 $x \cos \theta - y \sin \theta = d$.
- Basic Hough transform algorithm
 - Initialize $H[d, \theta] = 0$
 - For each edge point $I(x, y)$ in the image
for $\theta \in [\theta_{\min} \text{ to } \theta_{\max}]$ //some quantization
 $d = x \cos \theta - y \sin \theta$ $H[d, \theta] += 1$.
 - Find the value(s) of (d, θ) where $H[d, \theta]$ is maximum.
 - The detected line in the image is given
by $d = x \cos \theta - y \sin \theta$

H : accumulator array (votes)

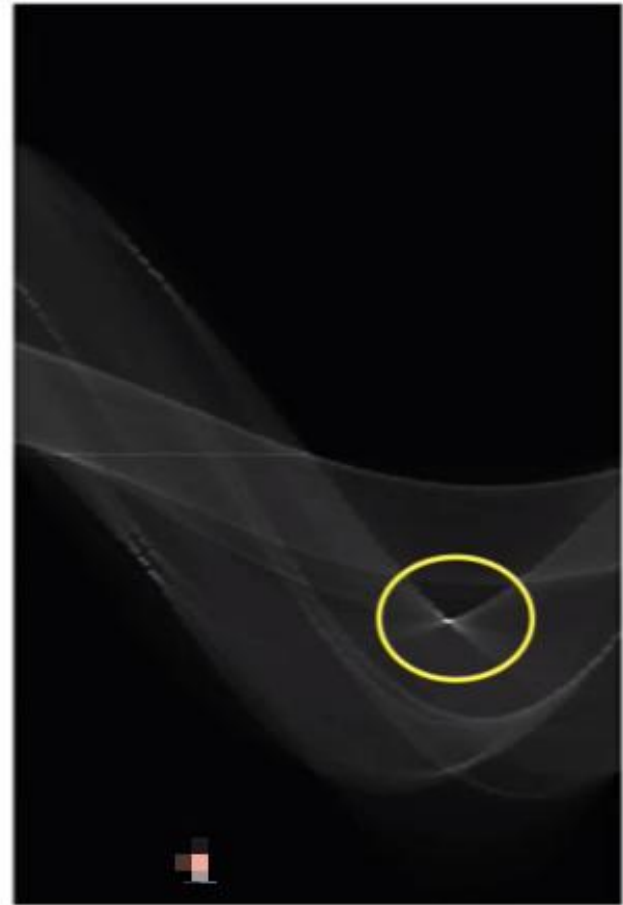
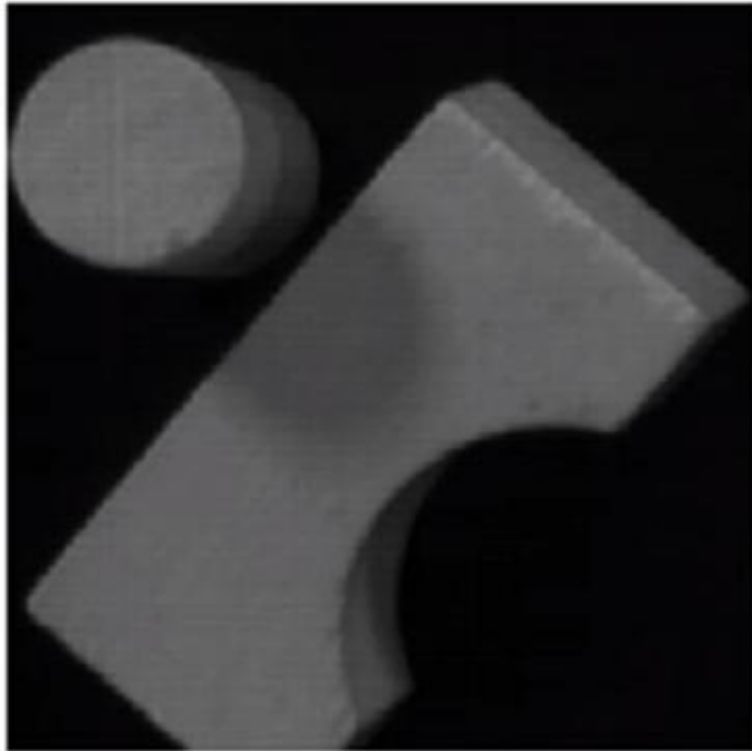


Time complexity (in terms of number of votes per pt)?

Example: Hough transform for straight lines

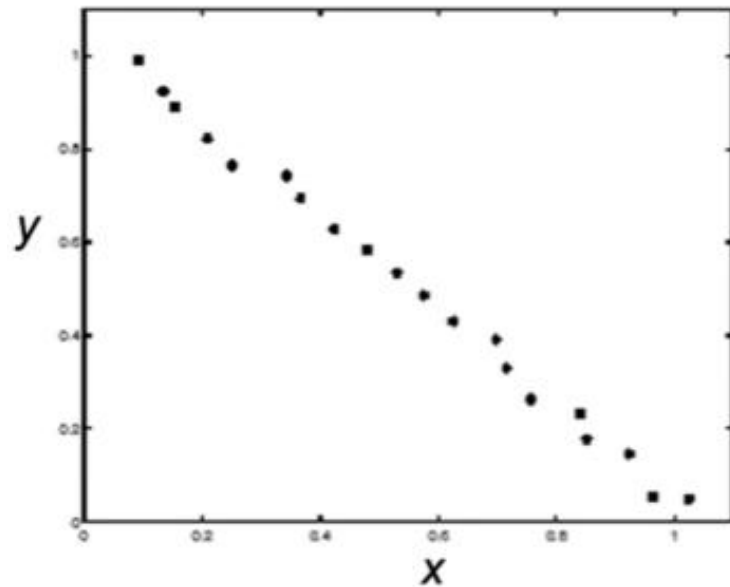


Example: Hough transform for straight lines

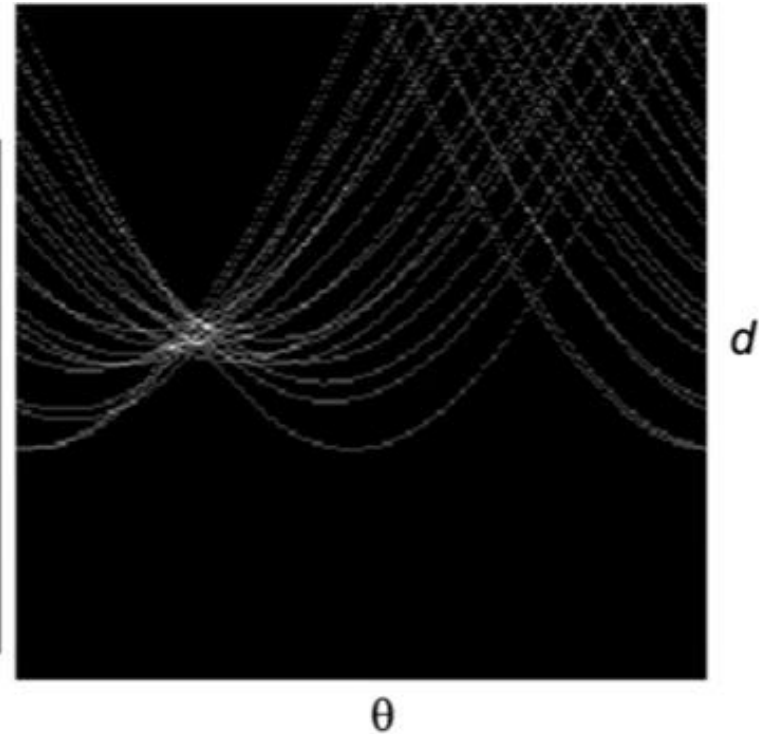


Which line generated this peak?

Impact of noise on Hough



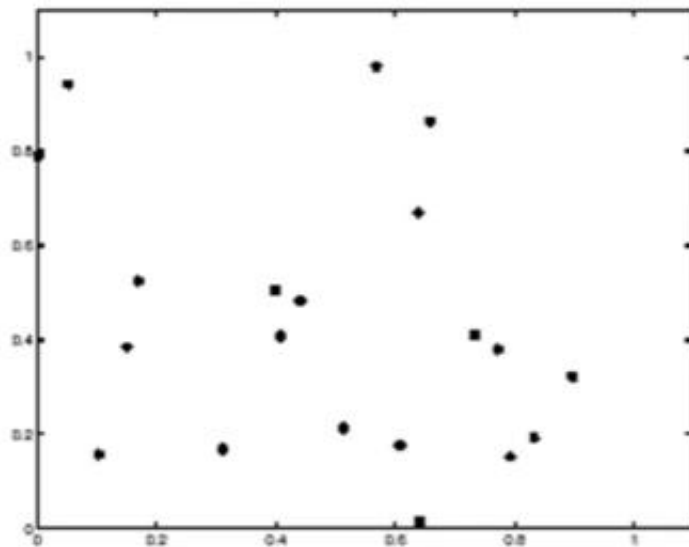
**Image space
edge coordinates**



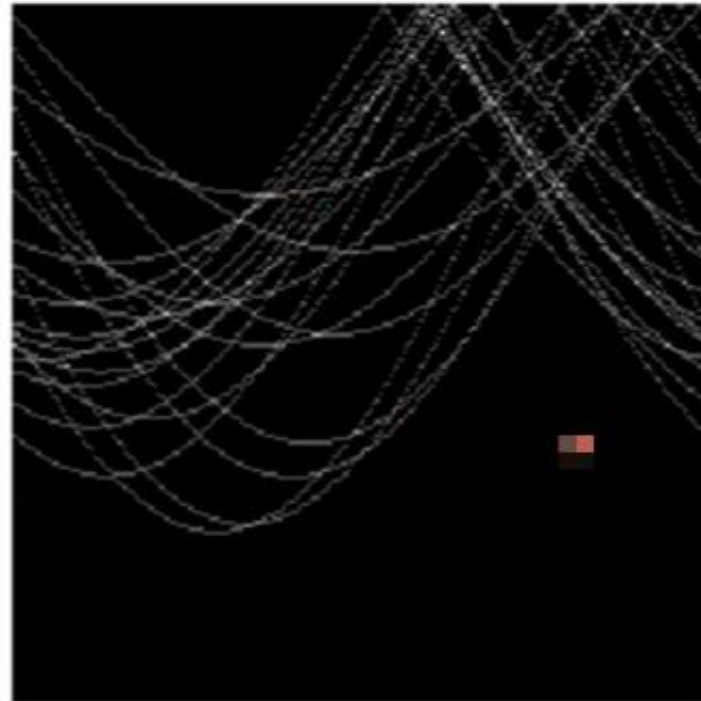
Votes

What difficulty does this present for an implementation?

Impact of noise on Hough



**Image space
edge coordinates**



Votes

Here, everything appears to be “noise”, or random edge points, but we still see peaks in the vote space

Dealing with noise

- Choose a good grid/discretization
 - **Too soarse**: large votes obtained when too many different lines correspond to a single bucket
 - **Too fine**: miss lines because some points that are not exactly collinear cast votes for different buckets
- Increment neighboring bins (smoothing in accumulator array)
- Try to get rid of irrelevant features
 - E.g., take only edge points with significant gradient magnitude

Extensions

- Extension 1: Use the image gradient

1. same

2. for each edge point $I[x, y]$ in the image

$\theta = \text{gradient at } (x, y)$

$$d = x \cos\theta - y \sin\theta$$

$$H[d, \theta]^+ = 1$$

3. same

4. same

(Reduces degrees of freedom)

Extensions

- **Extension 2:**

- Give more votes for stronger edges (use magnitude of gradient)

- **Extension 3:**

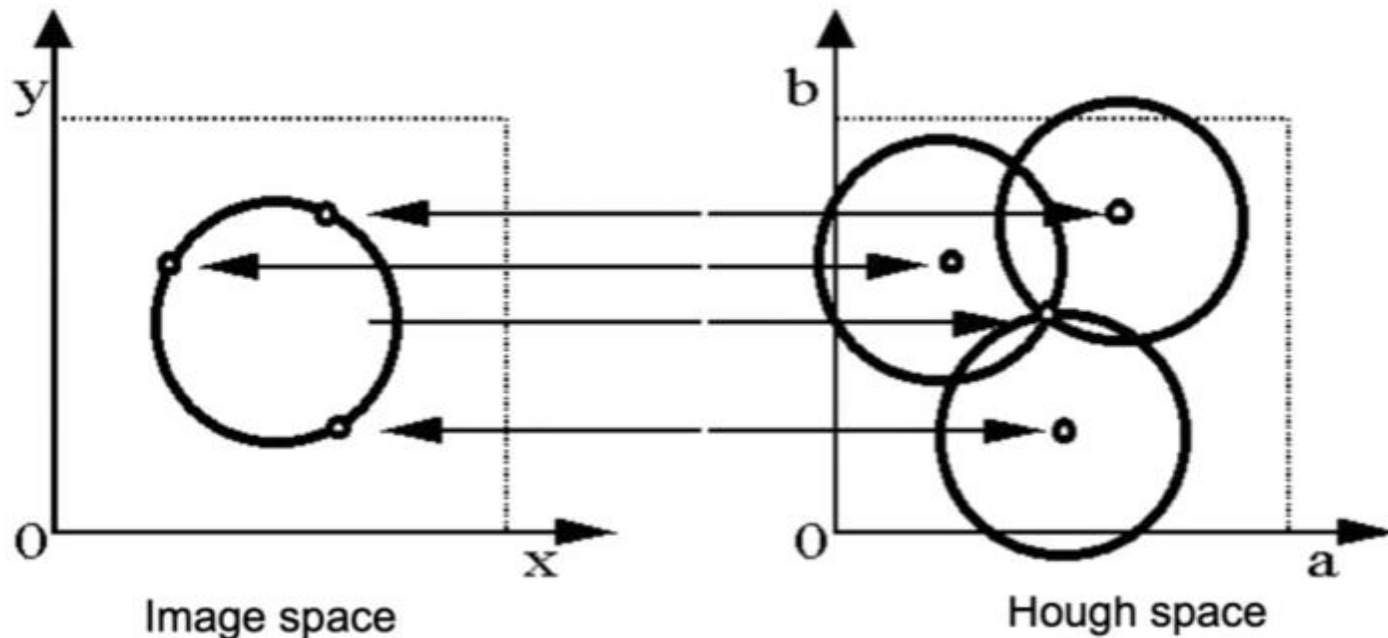
- change the sampling of (d, θ) to give more/less resolution.

- **Extension 4:**

- The same procedure can be used with circles, squares, or any other shape, ...

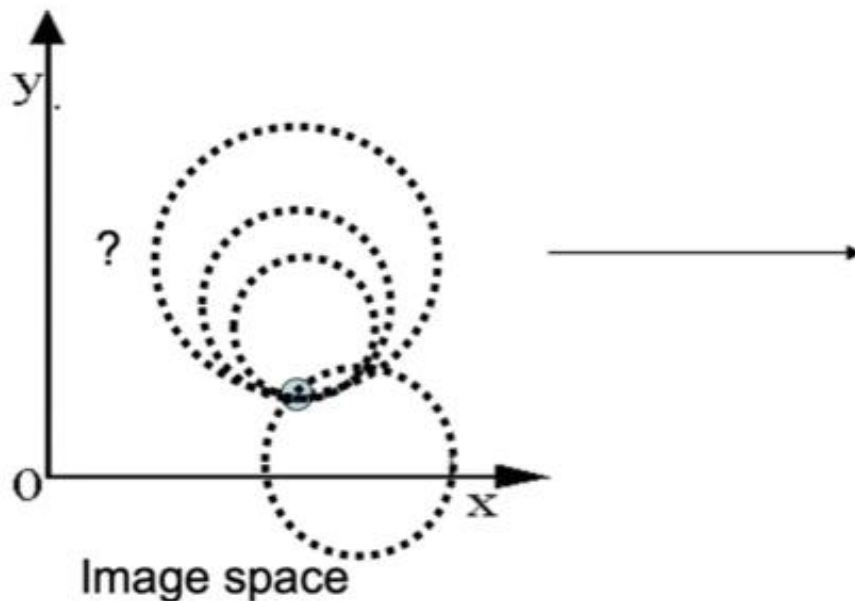
Hough transform for circles

- Circles: center (a, b) and radius r
$$(x_i - a)^2 + (y_i - b)^2 = r^2$$
- For a **fixed** radius r , **unknown** gradient directions



Hough transform for circles

- Circles: center (a, b) and radius r
$$(x_i - a)^2 + (y_i - b)^2 = r^2$$
- For an **unknown radius** r , **unknown** gradient directions



Hough transform for circles

- Circles: center (a, b) and radius r
$$(x_i - a)^2 + (y_i - b)^2 = r^2$$
- For an **unknown radius** r , **unknown** gradient directions

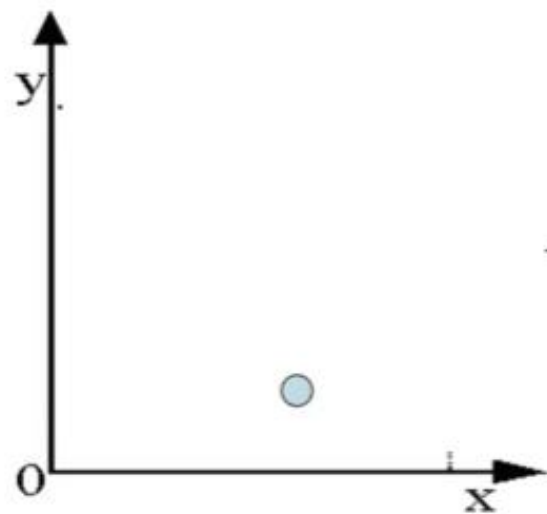
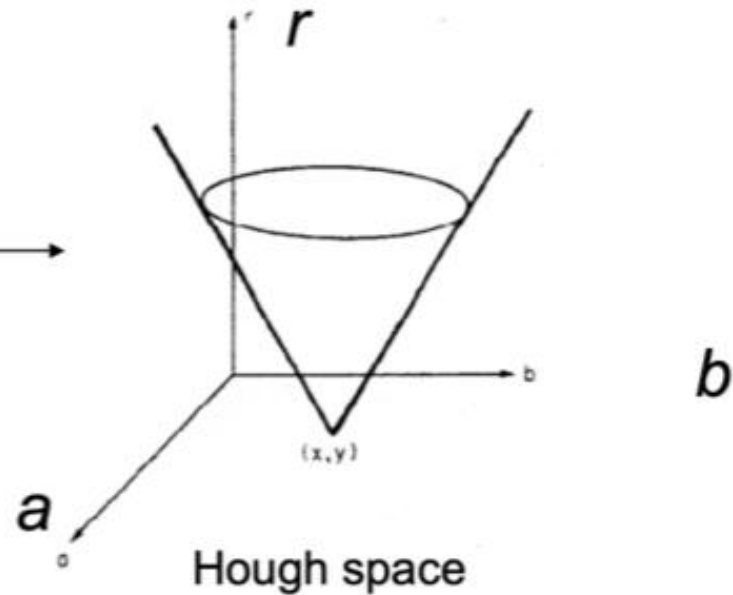
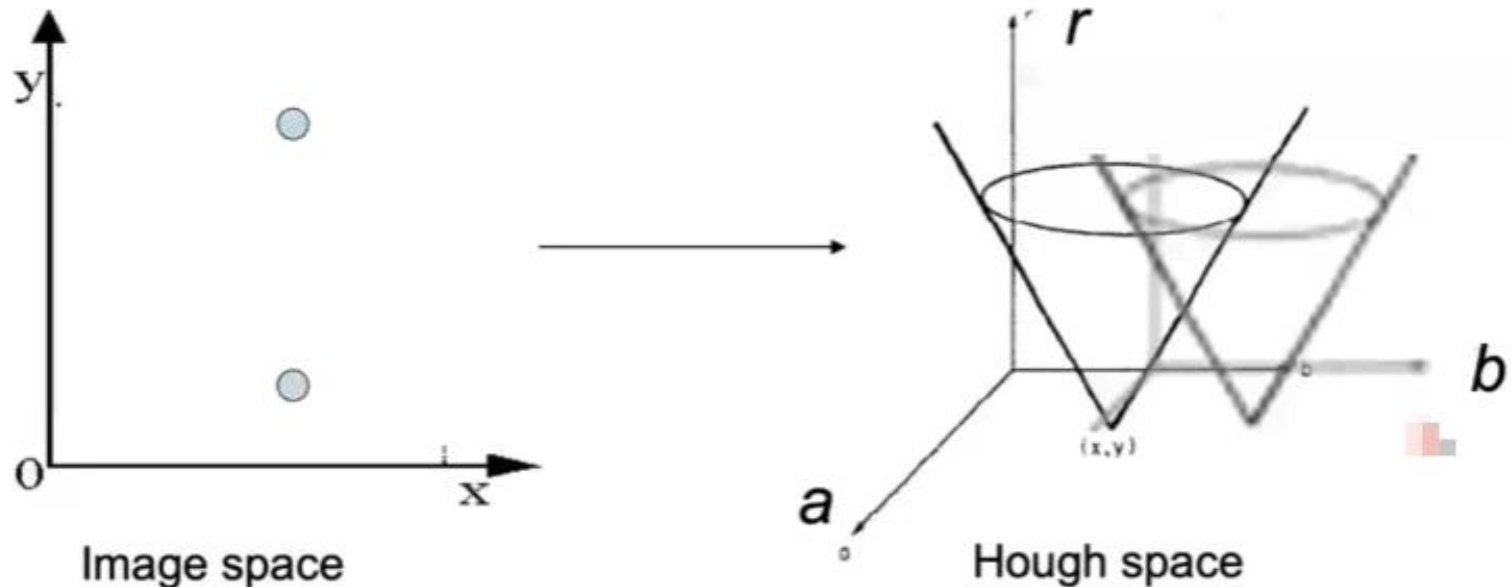


Image space



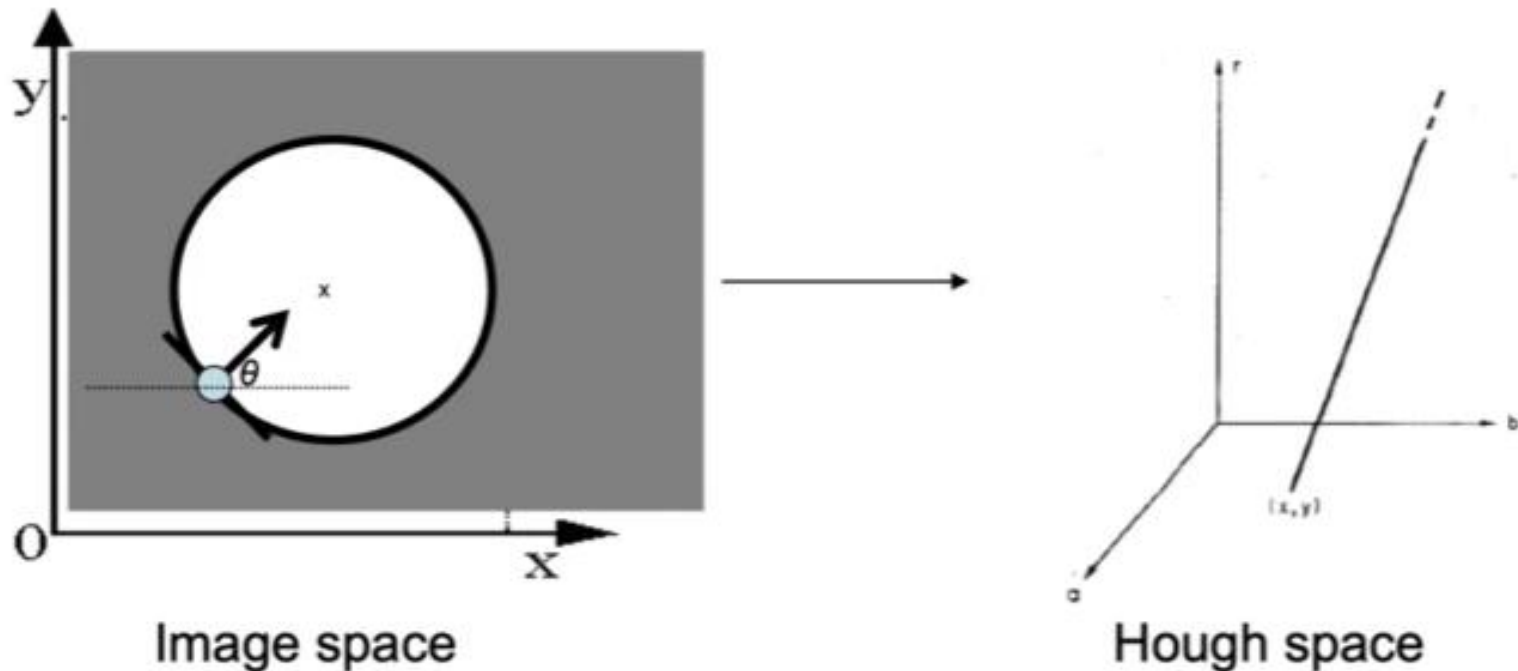
Hough transform for circles

- Circles: center (a, b) and radius r
$$(x_i - a)^2 + (y_i - b)^2 = r^2$$
- For an **unknown radius** r , **unknown** gradient directions



Hough transform for circles

- Circles: center (a, b) and radius r
$$(x_i - a)^2 + (y_i - b)^2 = r^2$$
- For an **unknown radius** r , **known** gradient directions



Hough transform for circles

- For every edge pixel (x, y) :

For each possible radius value r .

For each possible gradient direction θ

// or use estimated gradient at (x, y)

$a = x + r \cdot \cos\theta$ //column

$b = y - r \cdot \sin\theta$ //row

$H[a, b, r] + = 1$

end

end

Example: detecting circles with Hough

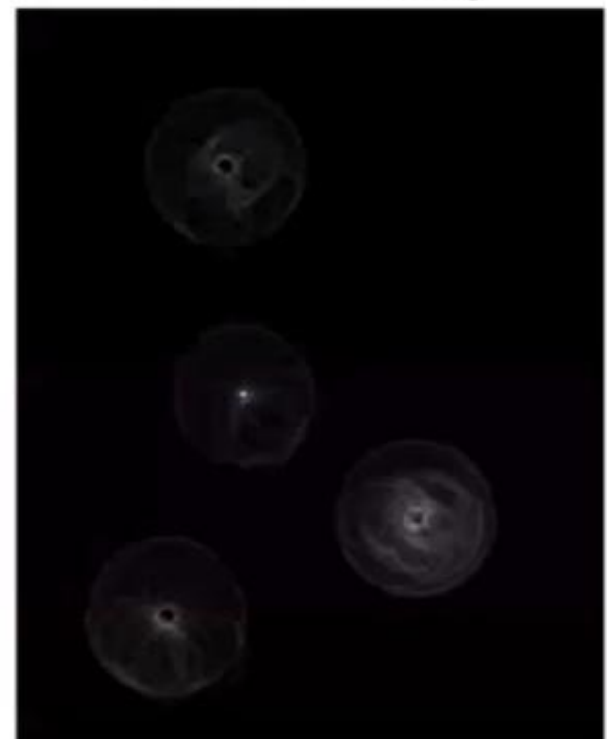
Original



Edges



Votes: Penny



Note: a different Hough transform (with separate accumulators) was used for each circle radius (quarters vs. penny)

Example: detecting circles with Hough

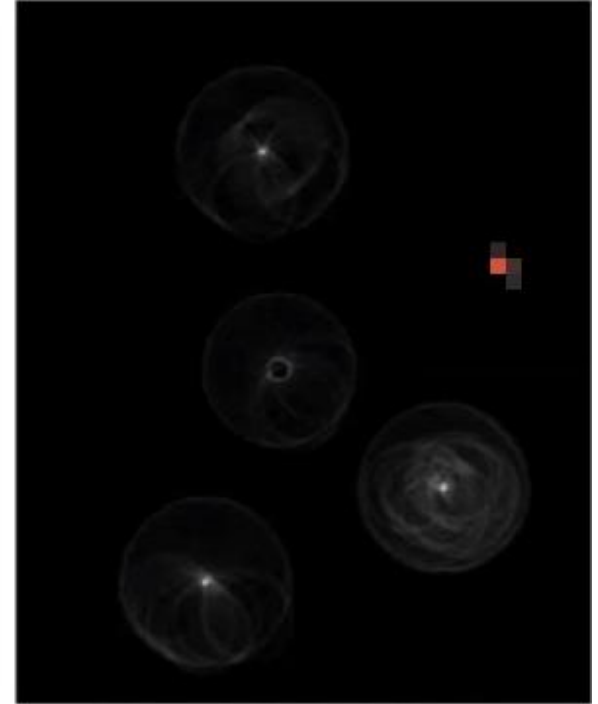
Original



Edges



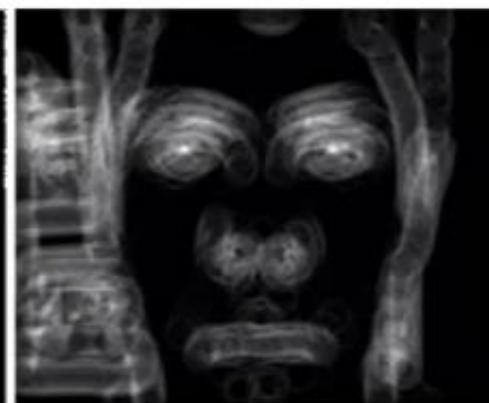
Votes: Quarter



Example: Iris detection



Gradient+threshold



Hough space
(fixed radius)

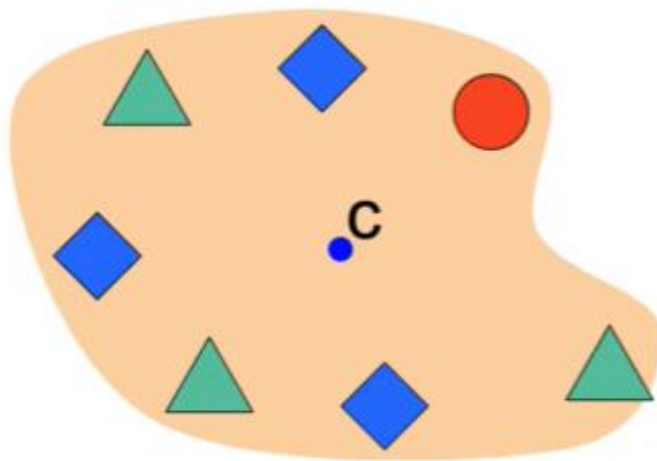


Max detections

Generalized Hough transform

- We want to find a template defined by its reference point (center) and several distinct types of landmark points in stable spatial configuration

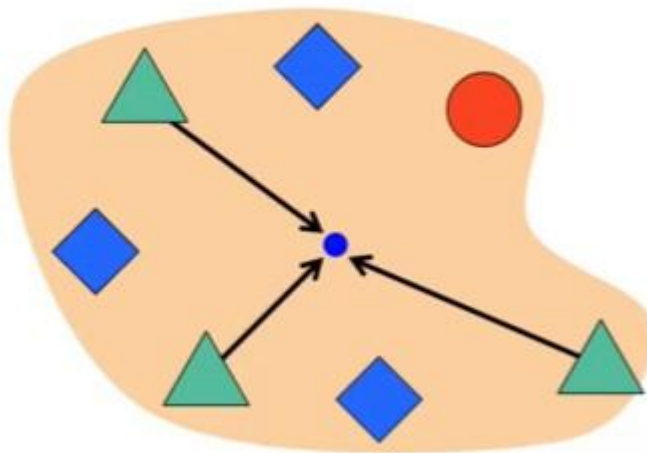
Template



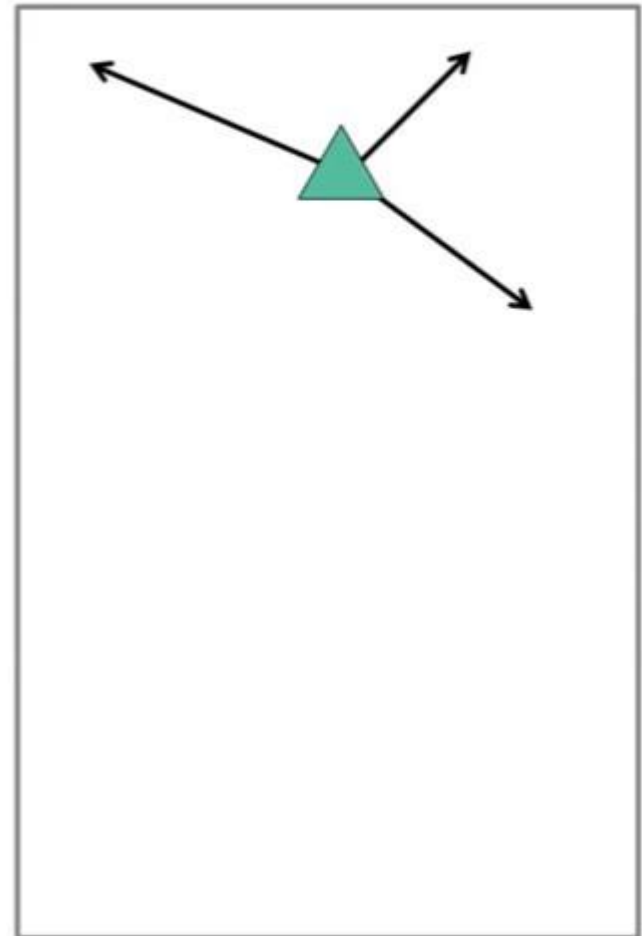
Generalized Hough transform

- Template representation: for each type that feature type in the model and vote for the possible center locations associated with that type in the model

Template



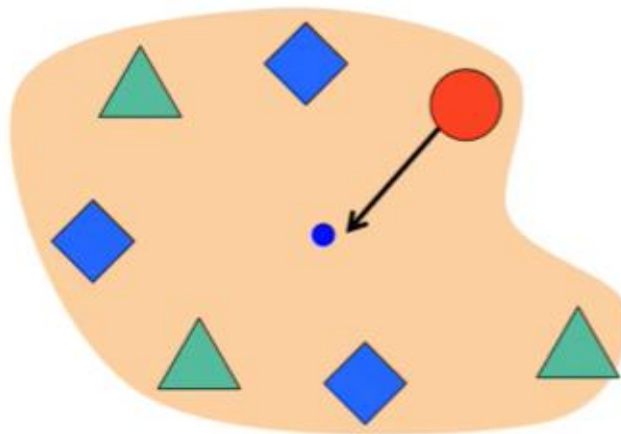
Model



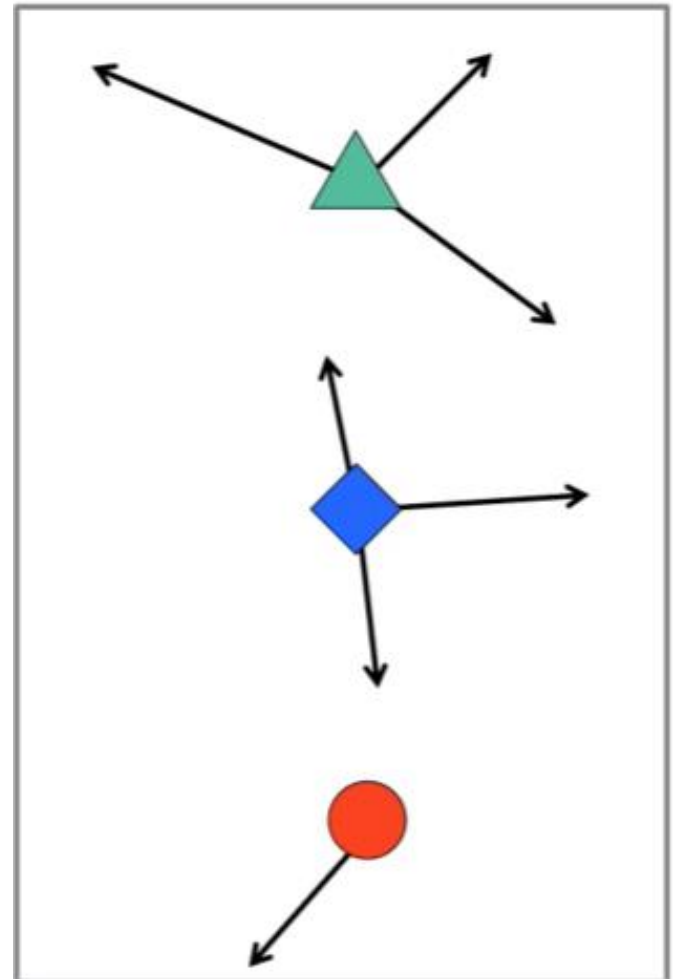
Generalized Hough transform

- Template representation: for each type that feature type in the model and vote for the possible center locations associated with that type in the model

Template



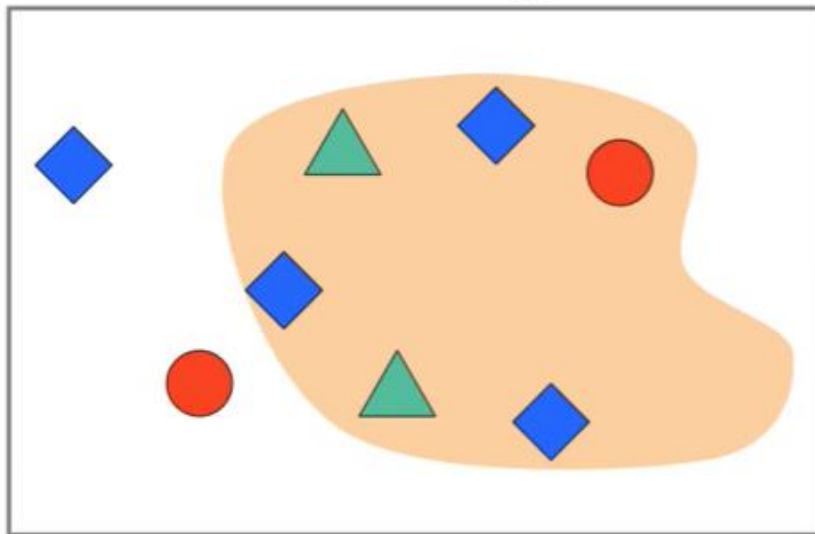
Model



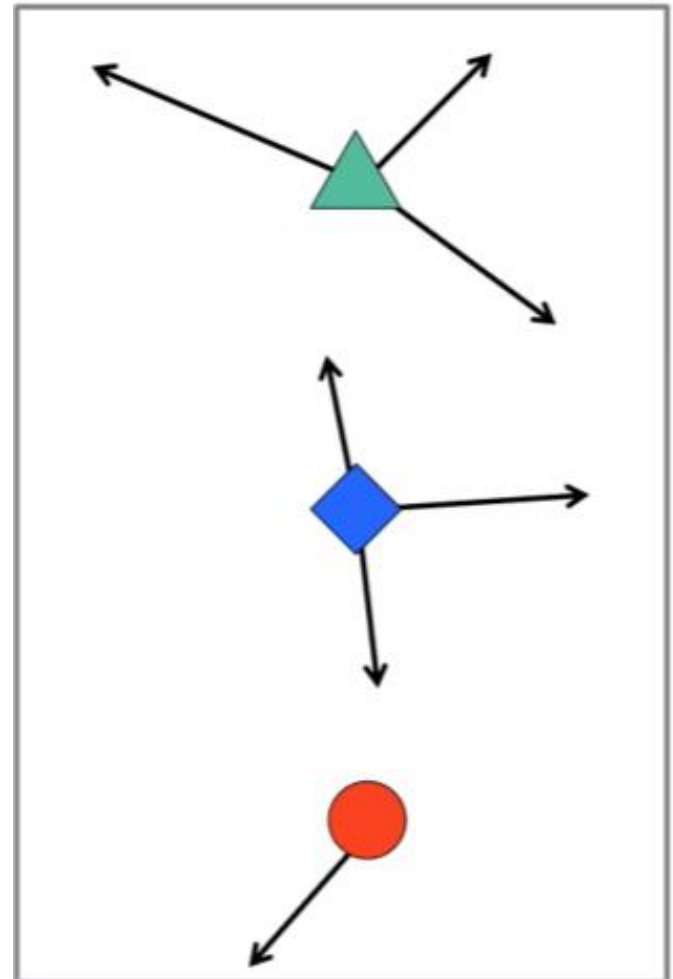
Generalized Hough transform

- Detecting the template: for each feature in a new image, look up that feature type in the model and vote for the possible center locations associated with that type in the model

Test image



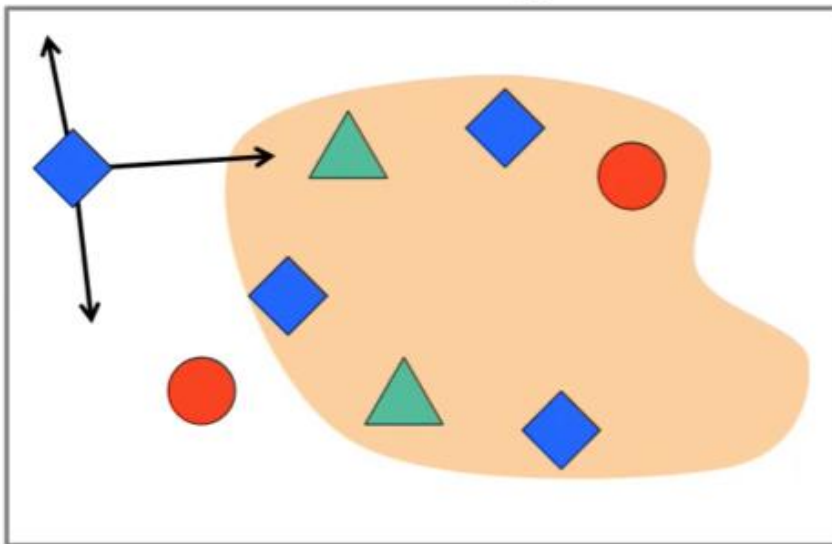
Model



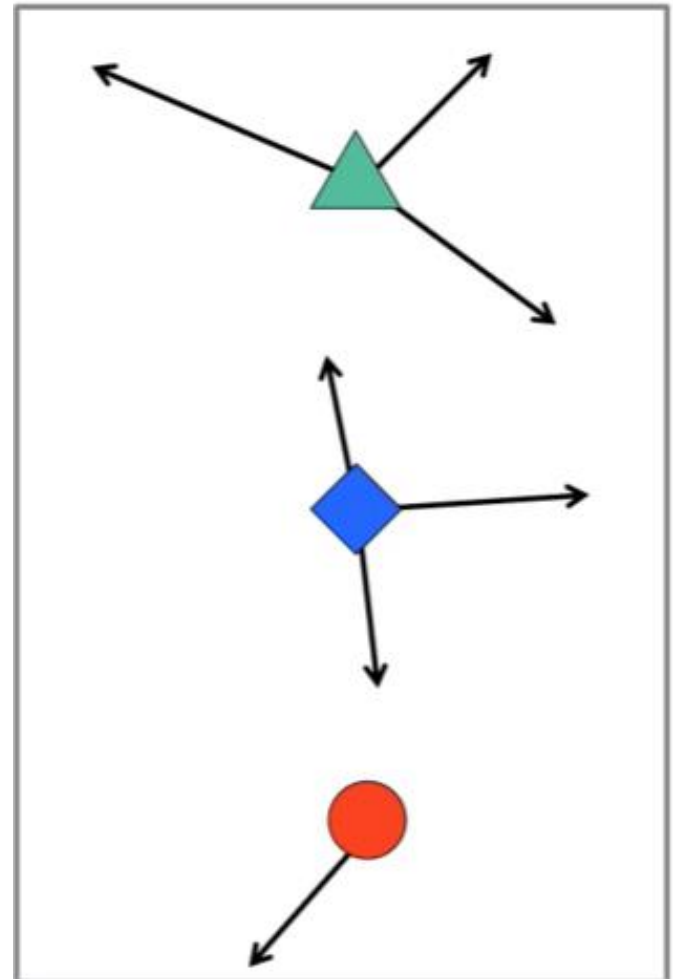
Generalized Hough transform

- Detecting the template: for each feature in a new image, look up that feature type in the model and vote for the possible center locations associated with that type in the model

Test image



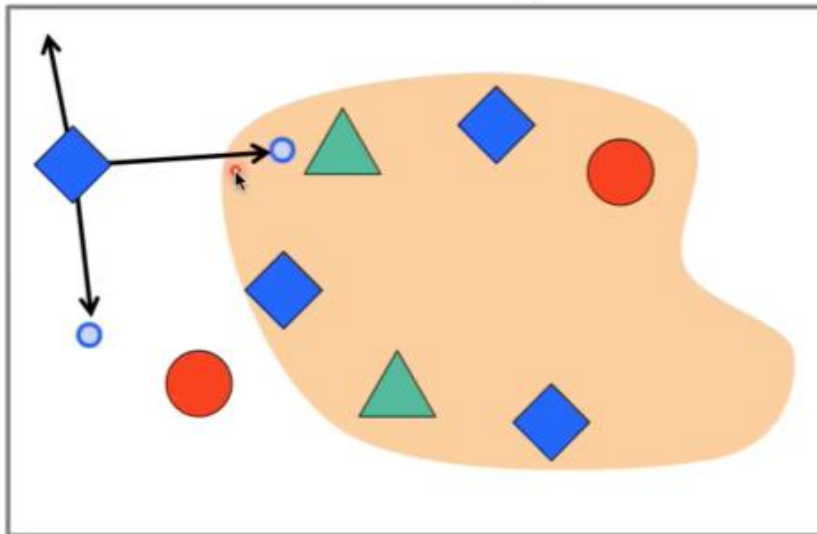
Model



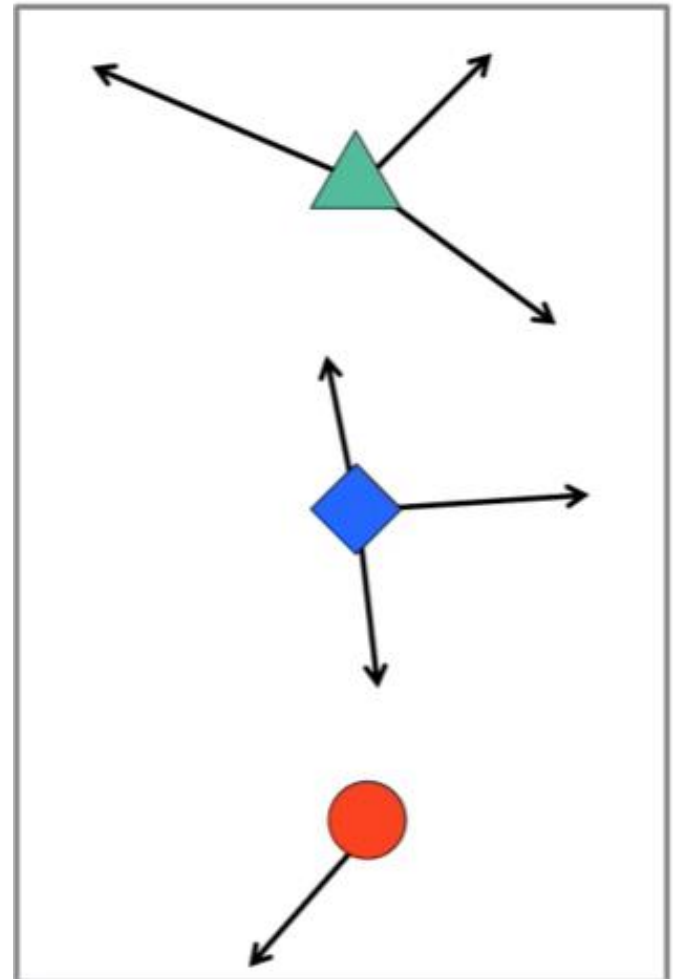
Generalized Hough transform

- Detecting the template: for each feature in a new image, look up that feature type in the model and vote for the possible center locations associated with that type in the model

Test image

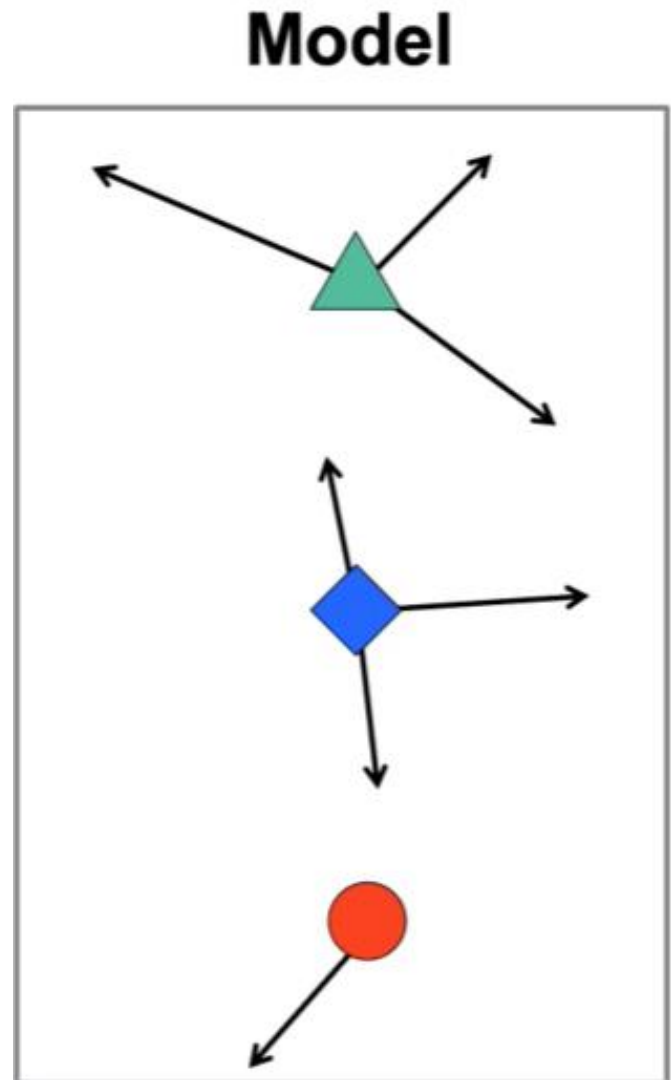
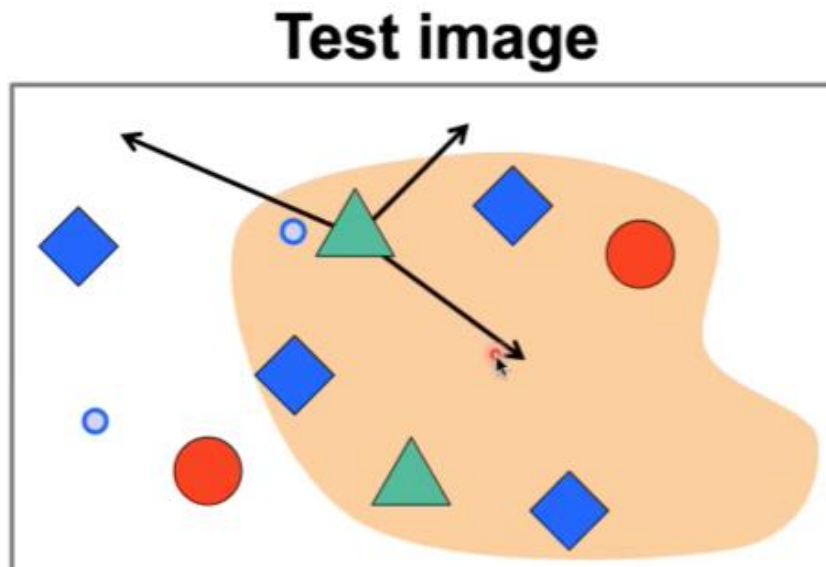


Model



Generalized Hough transform

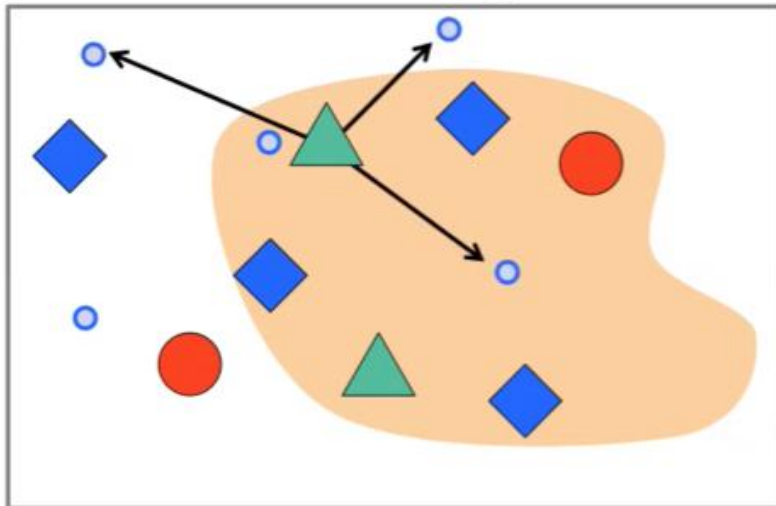
- Detecting the template: for each feature in a new image, look up that feature type in the model and vote for the possible center locations associated with that type in the model



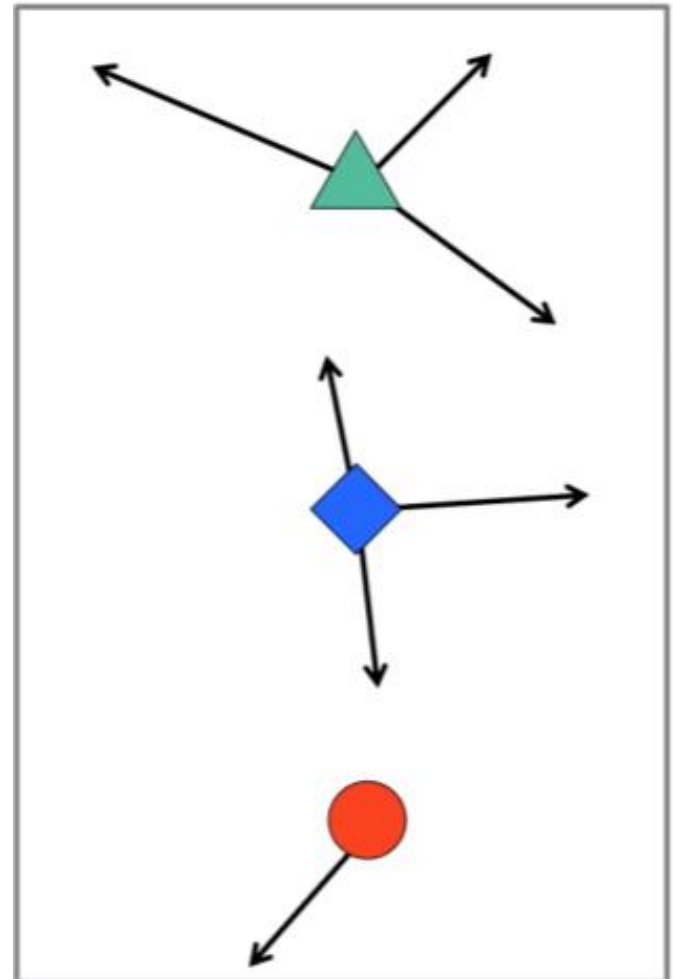
Generalized Hough transform

- Detecting the template: for each feature in a new image, look up that feature type in the model and vote for the possible center locations associated with that type in the model

Test image



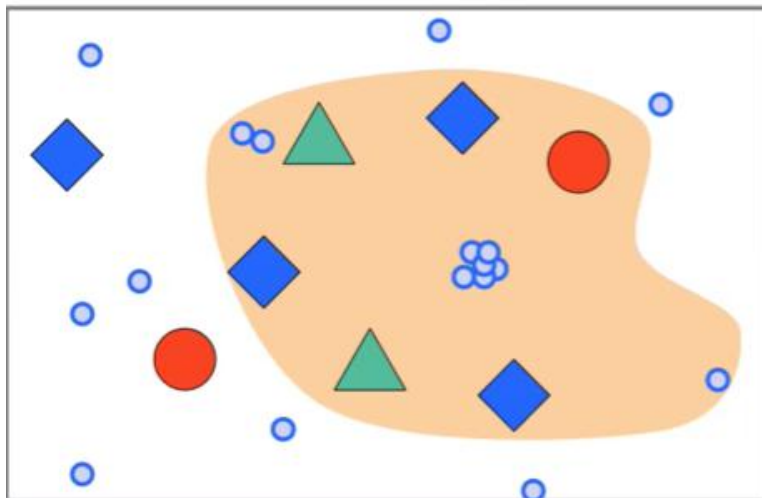
Model



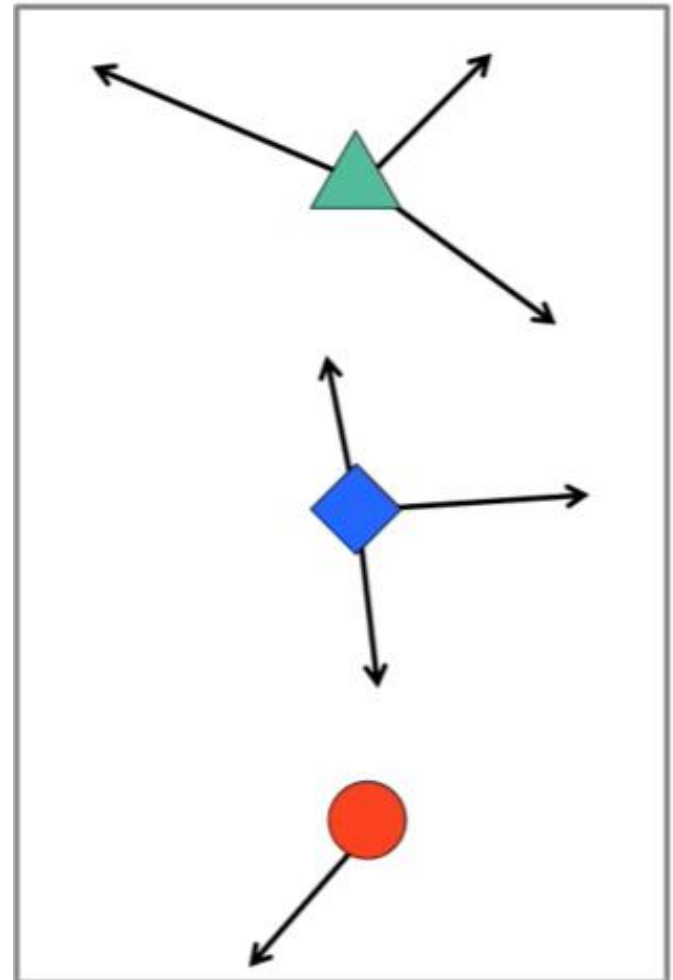
Generalized Hough transform

- Detecting the template: for each feature in a new image, look up that feature type in the model and vote for the possible center locations associated with that type in the model

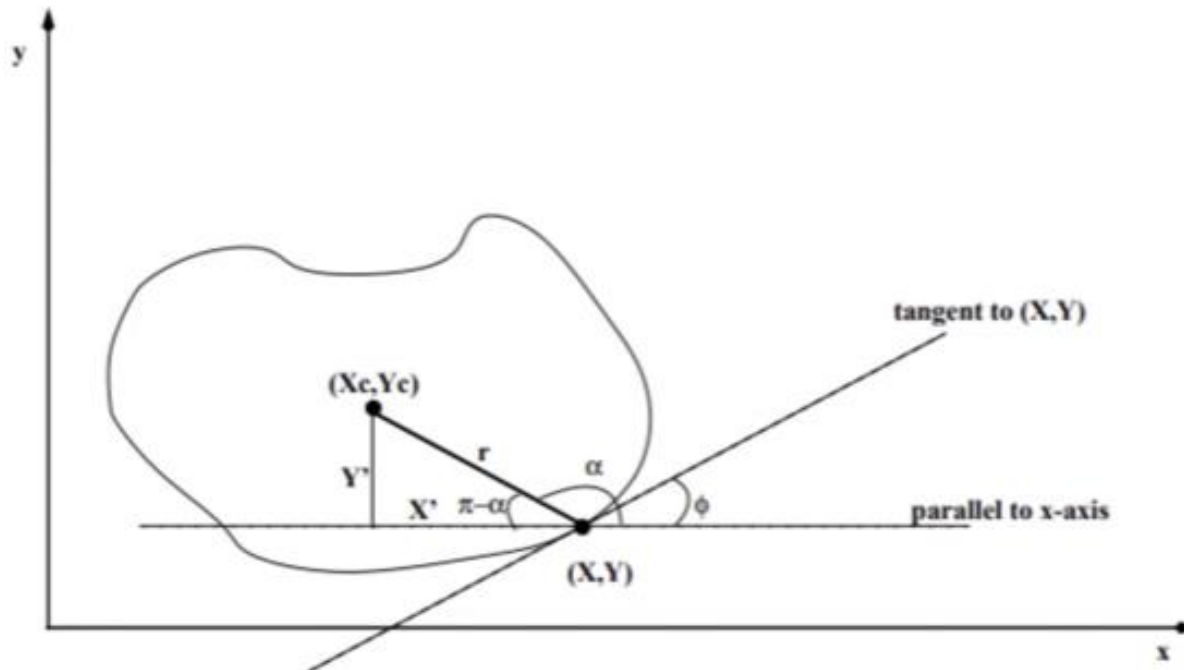
Test image



Model



Generalized Hough transform



$$\begin{aligned} x &= x_c + x' & \text{or} & & x_c &= x - x' \\ y &= y_c + y' & \text{or} & & y_c &= y - y' \end{aligned}$$

$$\begin{aligned} \cos(\pi - \alpha) &= \frac{y'}{r} & \text{or} & & y' &= r \cos(\pi - \alpha) = -r \sin(\alpha) \\ \sin(\pi - \alpha) &= \frac{x'}{r} & \text{or} & & x' &= r \sin(\pi - \alpha) = r \cos(\alpha) \end{aligned}$$

Generalized Hough transform

- Combining the above equations we have:

$$\begin{aligned}x_c &= x + r\cos(\alpha) \\ y_c &= y + r\sin(\alpha)\end{aligned}$$

Preprocessing step

- (1) Pick a reference point (e.g., (x_c, y_c))
- (2) Draw a line from the reference point to the boundary.
- (3) Compute ϕ (i.e., perpendicular to gradient's direction).
- (4) Store the reference point (x_c, y_c) as a function of ϕ (i.e., build the *R-table*)

$\phi_1:$	$(r_1^1, \alpha_1^1), (r_2^1, \alpha_2^1), \dots$
$\phi_2:$	$(r_1^2, \alpha_1^2), (r_2^2, \alpha_2^2), \dots$
$\vdots$	
$\phi_n:$	$(r_1^n, \alpha_1^n), (r_2^n, \alpha_2^n), \dots$

- The *R-table* allows us to use the contour edge points and gradient angle to recompute the location of the reference point.

Note: we need to build a separate *R-table* for each different object.

Generalized Hough transform

Detection

(1) Quantize the parameter space:

$$P[x_{c_{\min}} \cdots x_{c_{\max}}][y_{c_{\min}} \cdots y_{c_{\max}}]$$

(2) for each edge point (x, y)

(2.1) Using the gradient angle ϕ , retrieve from the *R-table* all the (α, r) values indexed under ϕ .

(2.2) For each (α, r) , compute the candidate reference points:

$$\begin{aligned}x_c &= x + r \cos(\alpha) \\ y_c &= y + r \sin(\alpha)\end{aligned}$$

(2.3) Increase counters (voting):

$$++(P[x_c][y_c])$$

(3) Possible locations of the object contour are given by local maxima in $P[x_c][y_c]$

- If $P[x_c][y_c] > T$, then the object contour is located at (x_c, y_c)

Concluding remarks

- **Advantages:**

- Conceptually simple
- Easy implementation
- Handles missing and occluded data very gracefully
- Can be adapted to many types of forms, not just lines

- **Disadvantages:**

- Computationally complex for objects with many parameters.
- Looks for only one single type of object
- The length and the position of a line segment can not be determined
- Co-linear line segments cannot be separated.